

```

1 // ===== frontend/src/components/app/pages/MultimodalModule.vue =====
2 <script setup lang="ts">
3 // @ts-nocheck
4 import { computed as vueComputed, isRef, unref } from "vue";
5 import type { PageMode } from "../../types";
6
7 const props = defineProps<{ app: Record<string, any> }>();
8 const app = props.app;
9 const bind = (target: Record<string, any>, key: string) =>
10   vueComputed({
11     get: () => unref(target[key]),
12     set: (value) => {
13       if (isRef(target[key])) target[key].value = value;
14       else target[key] = value;
15     },
16   });
17
18 const Cpu = bind(app, "Cpu");
19 const ChatDotRound = bind(app, "ChatDotRound");
20 const DataAnalysis = bind(app, "DataAnalysis");
21 const Delete = bind(app, "Delete");
22 const PictureFilled = bind(app, "PictureFilled");
23 const Promotion = bind(app, "Promotion");
24 const Search = bind(app, "Search");
25 const SwitchButton = bind(app, "SwitchButton");
26 const User = bind(app, "User");
27 const VideoCamera = bind(app, "VideoCamera");
28 const renderMarkdown = bind(app, "renderMarkdown");
29 const isAuthDialogOpen = bind(app, "isAuthDialogOpen");
30 const authUser = bind(app, "authUser");
31 const userInvitationCode = bind(app, "userInvitationCode");
32 const isAuthLoaded = bind(app, "isAuthLoaded");
33 const isComposing = bind(app, "isComposing");
34 const isAuthenticated = bind(app, "isAuthenticated");
35 const handleUserMenu = bind(app, "handleUserMenu");
36 const logout = bind(app, "logout");
37 const visionModels = bind(app, "visionModels");
38 const multimodalTab = bind(app, "multimodalTab");
39 const videoFile = bind(app, "videoFile");
40 const videoUrl = bind(app, "videoUrl");
41 const videoAnalysis = bind(app, "videoAnalysis");
42 const videoAnalysisRunning = bind(app, "videoAnalysisRunning");
43 const videoRef = bind(app, "videoRef");
44 const drivingCanvasRef = bind(app, "drivingCanvasRef");
45 const drivingRunning = bind(app, "drivingRunning");
46 const drivingSpeed = bind(app, "drivingSpeed");
47 const drivingSteering = bind(app, "drivingSteering");
48 const drivingAutoPilot = bind(app, "drivingAutoPilot");
49 const drivingStats = bind(app, "drivingStats");
50 const drivingScenario = bind(app, "drivingScenario");

```

```
1 const drivingWeather = bind(app, "drivingWeather");
2 const drivingControlMode = bind(app, "drivingControlMode");
3 const drivingOpenPilotMode = bind(app, "drivingOpenPilotMode");
4 const drivingScenarioOptions = bind(app, "drivingScenarioOptions");
5 const drivingControlModeOptions = bind(app, "drivingControlModeOptions");
6 const drivingPerception = bind(app, "drivingPerception");
7 const drivingOpenPilotState = bind(app, "drivingOpenPilotState");
8 const retrievalQuery = bind(app, "retrievalQuery");
9 const retrievalResults = bind(app, "retrievalResults");
10 const retrievalLoading = bind(app, "retrievalLoading");
11 const mmChatMessages = bind(app, "mmChatMessages");
12 const mmChatPrompt = bind(app, "mmChatPrompt");
13 const mmChatImages = bind(app, "mmChatImages");
14 const mmChatAudioBase64 = bind(app, "mmChatAudioBase64");
15 const mmChatAudioName = bind(app, "mmChatAudioName");
16 const mmChatVideoUrl = bind(app, "mmChatVideoUrl");
17 const mmChatVideoName = bind(app, "mmChatVideoName");
18 const mmChatMediaType = bind(app, "mmChatMediaType");
19 const mmChatFileRef = bind(app, "mmChatFileRef");
20 const mmChatAudioRef = bind(app, "mmChatAudioRef");
21 const mmChatVideoRef = bind(app, "mmChatVideoRef");
22 const isMmChatSubmitting = bind(app, "isMmChatSubmitting");
23 const drivingAiAnalyzing = bind(app, "drivingAiAnalyzing");
24 const drivingAiAnalysis = bind(app, "drivingAiAnalysis");
25 const drivingAiIntervalSec = bind(app, "drivingAiIntervalSec");
26 const multimodalModel = bind(app, "multimodalModel");
27 const openImageInNewTab = bind(app, "openImageInNewTab");
28 const startDrivingSim = bind(app, "startDrivingSim");
29 const stopDrivingSim = bind(app, "stopDrivingSim");
30 const handleVideoUpload = bind(app, "handleVideoUpload");
31 const runVideoAnalysis = bind(app, "runVideoAnalysis");
32 const stopVideoAnalysis = bind(app, "stopVideoAnalysis");
33 const searchImages = bind(app, "searchImages");
34 const handleMmImageUpload = bind(app, "handleMmImageUpload");
35 const handleMmAudioUpload = bind(app, "handleMmAudioUpload");
36 const handleMmVideoUpload = bind(app, "handleMmVideoUpload");
37 const removeMmMedia = bind(app, "removeMmMedia");
38 const submitMmChat = bind(app, "submitMmChat");
39 const stopMmChat = bind(app, "stopMmChat");
40 const clearMmChat = bind(app, "clearMmChat");
41 const analyzeDrivingScene = bind(app, "analyzeDrivingScene");
42 const stopDrivingAnalysis = bind(app, "stopDrivingAnalysis");
43 const copyToClipboard = bind(app, "copyToClipboard");
44 const openVideoUploadInput = bind(app, "openVideoUploadInput");
45 </script>
46
47 <template>
48   <div class="page-header">
49     <div class="header-left" style="gap: 8px">
50       <strong>全模态 Beta</strong>
```

```

1  <el-select
2    v-model="multimodalModel"
3    filterable
4    placeholder="视觉模型"
5    style="width: 200px"
6    size="small"
7  >
8    <el-option
9      v-for="m in visionModels"
10     :key="m.id"
11     :label="m.id"
12     :value="m.id"
13   />
14 </el-select>
15 </div>
16 <div class="header-right">
17   <template v-if="isAuthLoaded">
18     <el-button
19       v-if="!isAuthenticated"
20       type="primary"
21       plain
22       @click="isAuthDialogOpen = true"
23     >登录 / 注册</el-button>
24   >
25   <el-dropdown v-else trigger="click" @command="handleUserMenu">
26     <el-tag type="success" style="cursor: pointer"
27       >{{ authUser?.username }} </el-tag>
28   >
29   <template #dropdown>
30     <el-dropdown-menu>
31       <el-dropdown-item command="console"
32         ><el-icon><DataAnalysis /></el-icon> 控制台</el-dropdown-item>
33     >
34     <el-dropdown-item v-if="userInvitationCode" disabled>
35       <span style="color: #666"
36         >邀请码: {{ userInvitationCode }}</span>
37     >
38     <el-button
39       size="small"
40       style="margin-left: 8px"
41       @click.stop="copyToClipboard(userInvitationCode)"
42     >复制</el-button>
43   >
44 </el-dropdown-item>
45   <el-dropdown-item command="logout" divided
46     ><el-icon><SwitchButton /></el-icon> 退出登录</el-dropdown-item>
47   >
48 </el-dropdown-menu>
49 </template>
50 </el-dropdown>

```

```

1   </template>
2   </div>
3   </div>
4
5   <div class="multimodal-page">
6     <el-tabs v-model="multimodalTab" type="border-card" class="mm-tabs">
7       <!-- ===== Tab 1: Autonomous Driving Simulation ===== -->
8       <el-tab-pane label="自动驾驶模拟" name="driving">
9         <div class="driving-layout">
10          <div class="driving-main">
11            <div class="driving-canvas-wrap">
12              <canvas ref="drivingCanvasRef" class="driving-canvas" />
13              <div v-if="!drivingRunning" class="driving-overlay">
14                <el-icon :size="48" color="#fff"><VideoCamera /></el-icon>
15                <span>点击下方「启动仿真」开始</span>
16              </div>
17            </div>
18          <div class="driving-hud">
19            <div class="hud-item">
20              <span class="hud-label">速度</span>
21              <span class="hud-value" style="color: #3b82f6">
22                <{{ Math.round(drivingSpeed) }}> <small>km/h</small></span>
23              </div>
24            </div>
25            <div class="hud-item">
26              <span class="hud-label">FPS</span>
27              <span class="hud-value" style="color: #22c55e">{{
28                drivingStats.fps
29              }}</span>
30            </div>
31            <div class="hud-item">
32              <span class="hud-label">检测目标</span>
33              <span class="hud-value" style="color: #f59e0b">{{
34                drivingStats.objects
35              }}</span>
36            </div>
37            <div class="hud-item">
38              <span class="hud-label">车道偏离</span>
39              <span
40                class="hud-value"
41                :style="{
42                  color:
43                    Math.abs(drivingStats.laneDev) > 30
44                      ? '#ef4444'
45                      : '#8b5cf6',
46                }"
47              >{{ drivingStats.laneDev }} <small>px</small></span>
48            </div>
49          </div>
50        <div class="hud-item">

```

```

1      <span class="hud-label">前车距离</span>
2      <span
3          class="hud-value"
4          :style="{ color: drivingStats.fcw ? '#ef4444' : '#14b8a6' }"
5          >{{
6              drivingStats.leadDistance >= 999
7              ? "∞"
8              : drivingStats.leadDistance
9          }}
10         <small>m</small></span>
11     >
12 </div>
13 <div class="hud-item">
14     <span class="hud-label">TTC</span>
15     <span
16         class="hud-value"
17         :style="{
18             color:
19                 drivingStats.ttc < 3
20                 ? '#ef4444'
21                 : drivingStats.ttc < 5
22                 ? '#f59e0b'
23                 : '#22c55e',
24             }"
25         >{{ drivingStats.ttc >= 99 ? "∞" : drivingStats.ttc }}
26         <small>s</small></span>
27     >
28 </div>
29 </div>
30 </div>
31 <div class="driving-panel">
32     <el-card shadow="never" class="driving-card">
33         <template #header>
34             <div
35                 style="
36                     display: flex;
37                     align-items: center;
38                     justify-content: space-between;
39                     gap: 8px;
40                 "
41             >
42                 <strong>仿真控制</strong>
43                 <el-tag size="small" type="success">激进 openpilot</el-tag>
44             </div>
45         </template>
46         <div class="driving-controls">
47             <div class="control-row">
48                 <span>OpenPilot 场景</span>
49                 <el-select
50                     v-model="drivingScenario"

```

```

1      style="width: 100%"
2      :disabled="drivingRunning"
3  >
4      <el-option
5          v-for="item in drivingScenarioOptions"
6          :key="item.value"
7          :label="item.label"
8          :value="item.value"
9      />
10 </el-select>
11 </div>
12 <div class="control-row">
13     <span>天气/能见度</span>
14     <el-segmented
15         v-model="drivingWeather"
16         :options="[
17             { label: '晴', value: 'clear' },
18             { label: '雨', value: 'rain' },
19             { label: '雾', value: 'fog' },
20             { label: '夜', value: 'night' },
21         ]"
22     />
23 </div>
24 <div class="control-row">
25     <span>速度 (km/h)</span>
26     <el-slider
27         v-model="drivingSpeed"
28         :min="0"
29         :max="150"
30         :step="5"
31         show-input
32         :disabled="!drivingRunning"
33     />
34 </div>
35 <div class="control-row">
36     <span>转向</span>
37     <el-slider
38         v-model="drivingSteering"
39         :min="-1"
40         :max="1"
41         :step="0.05"
42         show-input
43         :disabled="drivingAutoPilot"
44     />
45 </div>
46 <div class="control-row">
47     <span>自动驾驶</span>
48     <el-switch v-model="drivingAutoPilot" />
49 </div>
50 </div class="control-row">

```

```

1      <span>控制模式</span>
2      <el-radio-group
3          v-model="drivingControlMode"
4          class="driving-mode-group"
5      >
6          <el-radio-button
7              v-for="item in drivingControlModeOptions"
8              :key="item.value"
9              :label="item.value"
10         >
11             {{ item.label }}
12         </el-radio-button>
13     </el-radio-group>
14 </div>
15 <div class="control-row">
16     <span>AI 分析间隔 ({{ drivingAiIntervalSec }}秒)</span>
17     <el-slider
18         v-model="drivingAiIntervalSec"
19         :min="1"
20         :max="10"
21         :step="1"
22         show-input
23     />
24 </div>
25 <div class="driving-model-card">
26     <span>当前分析模型</span>
27     <strong>{{ multimodalModel || "未选择" }}</strong>
28 </div>
29 <div class="driving-perception-card" :class="drivingStats.risk">
30     <strong>{{ drivingPerception.alert }}</strong>
31     <span
32         >Lane confidence {{ drivingPerception.laneConfidence }} ·
33         Lead {{ drivingPerception.leadDistance }}m · TTC
34         {{ drivingPerception.ttc }}s</span>
35     >
36 </div>
37 <div class="openpilot-state-grid">
38     <div>
39         <span>LongControl</span>
40         ><strong>{{ drivingOpenPilotState.longState }}</strong>
41     </div>
42     <div>
43         <span>LaneChange</span>
44         ><strong>{{
45             drivingOpenPilotState.laneChangeState
46         }}</strong>
47     </div>
48     <div>
49         <span>Accel</span>
50         ><strong>

```

```
1      >{{ drivingOpenPilotState.outputAccel }} m/s </strong>
2      >
3    </div>
4    <div>
5      <span>Lat PID</span>
6      ><strong>{{ drivingOpenPilotState.steerPid }}</strong>
7    </div>
8  </div>
9  <div class="control-actions">
10    <el-button
11      v-if="!drivingRunning"
12      type="primary"
13      :icon="VideoCamera"
14      @click="startDrivingSim()"
15    >启动仿真</el-button>
16    >
17    <el-button
18      v-else
19      type="danger"
20      :icon="SwitchButton"
21      @click="stopDrivingSim()"
22    >停止仿真</el-button>
23    >
24  </div>
25 </div>
26 </el-card>
27 <el-card shadow="never" class="driving-card">
28   <template #header>
29     <div
30       style="
31         display: flex;
32         align-items: center;
33         justify-content: space-between;
34       "
35     >
36       <strong>AI 场景分析</strong>
37     <el-button
38       v-if="drivingRunning && !drivingAiAnalyzing"
39       size="small"
40       type="primary"
41       :disabled="drivingControlMode === 'openpilot'"
42       @click="analyzeDrivingScene()"
43     >
44       分析当前场景
45     </el-button>
46     <el-button
47       v-if="drivingAiAnalyzing"
48       size="small"
49       type="danger"
50       @click="stopDrivingAnalysis()"
```



```

1      >停止</el-button
2
3    </div>
4  </template>
5  <div class="driving-analysis">
6    <div
7      v-if="!drivingAiAnalysis && !drivingAiAnalyzing"
8      style="
9        color: #909399;
10       font-size: 13px;
11       text-align: center;
12       padding: 20px 0;
13     "
14   >
15     {{
16       drivingControlMode === "openpilot"
17       ? "纯 openpilot 模式不调用视觉模型"
18       : "点击「分析当前场景」调用视觉模型分析驾驶画面"
19     }}
20   </div>
21   <div
22     v-else-if="drivingAiAnalyzing && !drivingAiAnalysis"
23     style="
24       color: #909399;
25       font-size: 13px;
26       text-align: center;
27       padding: 20px 0;
28     "
29   >
30     正在调用视觉模型分析驾驶场景...
31   </div>
32   <pre v-else class="driving-decision-output">{{
33     drivingAiAnalysis
34   }}</pre>
35 </div>
36 </el-card>
37 </div>
38 </div>
39 </el-tab-pane>
40
41 <!-- ===== Tab 2: Video Understanding ===== -->
42 <el-tab-pane label="视频理解" name="video">
43   <div class="video-layout">
44     <div class="video-player-section">
45       <el-card shadow="never">
46         <template #header><strong>视频播放</strong></template>
47         <div
48           v-if="!videoUrl"
49           class="video-upload-area"
50           @click="openVideoUploadInput()"

```

```

1      >
2      <el-icon :size="48" color="#c0c4cc"><VideoCamera /></el-icon>
3      <p>点击上传视频文件 (MP4, WebM)</p>
4      <input
5        id="video-upload-input"
6        type="file"
7        accept="video/*"
8        style="display: none"
9        @change="handleVideoUpload"
10     />
11   </div>
12   <div v-else class="video-player-wrap">
13     <video
14       ref="videoRef"
15       :src="videoUrl"
16       controls
17       style="width: 100%; max-height: 360px; border-radius: 8px"
18     />
19     <div style="margin-top: 8px; display: flex; gap: 8px">
20       <el-button
21         size="small"
22         @click="
23           videoUrl = '';
24           videoFile = null;
25           videoAnalysis = [];
26           "
27       >清除</el-button>
28     >
29     <el-button
30       v-if="!videoAnalysisRunning"
31       size="small"
32       type="primary"
33       @click="runVideoAnalysis()"
34     >开始分析</el-button>
35   >
36   <el-button
37     v-else
38     size="small"
39     type="danger"
40     @click="stopVideoAnalysis()"
41   >停止分析</el-button>
42   >
43   </div>
44 </div>
45 </el-card>
46 </div>
47 <div class="video-results-section">
48   <el-card shadow="never">
49     <template #header>
50     <div

```

```

1      style="
2      display: flex;
3      align-items: center;
4      justify-content: space-between;
5      "
6  >
7    <strong>分析结果</strong>
8    <el-tag
9      v-if="videoAnalysisRunning"
10     size="small"
11     type="warning"
12   >分析中...</el-tag>
13  >
14 </div>
15 </template>
16 <div
17   v-if="videoAnalysis.length === 0"
18   style="
19     color: #909399;
20     font-size: 13px;
21     text-align: center;
22     padding: 40px 0;
23   "
24   >
25     <el-icon :size="36"><DataAnalysis /></el-icon>
26     <p>尚未进行分析，请先上传视频并点击"开始分析"</p>
27   </div>
28   <el-scrollbar v-else max-height="360px">
29     <div
30       v-for="(item, idx) in videoAnalysis"
31       :key="idx"
32       class="video-result-item"
33     >
34       <div class="vr-header">
35         <span class="vr-time">{{ item.time.toFixed(1) }}s</span>
36         <el-tag
37           size="small"
38           :type="item.confidence > 0.85 ? 'success' : 'warning'"
39         >{{ item.label }}</el-tag>
40       >
41       <span class="vr-conf"
42         >{{ (item.confidence * 100).toFixed(0) }}%</span>
43     >
44   </div>
45   <div v-if="item.bbox" class="vr-bbox">
46     bbox: [{{ item.bbox.map((v) => Math.round(v)).join(", ") }}]
47   </div>
48 </div>
49 </el-scrollbar>
50 </el-card>

```

```

1     </div>
2 </div>
3 </el-tab-pane>
4
5 <!-- ===== Tab 3: Image-Text Retrieval ===== -->
6 <el-tab-pane label="图文检索" name="retrieval">
7   <div class="retrieval-section">
8     <div class="retrieval-search">
9       <el-input
10         v-model="retrievalQuery"
11         placeholder="输入检索关键词，如：城市交通、自动驾驶场景、道路标识..."
12         size="large"
13         @keyup.enter="searchImages()"
14       >
15         <template #append>
16           <el-button
17             :icon="Search"
18             :loading="retrievalLoading"
19             @click="searchImages()"
20           >检索</el-button>
21         >
22       </template>
23     </el-input>
24   </div>
25   <div
26     v-if="retrievalResults.length === 0 && !retrievalLoading"
27     class="retrieval-empty"
28   >
29     <el-empty description="输入关键词开始图文检索" :image-size="96">
30       <template #image>
31         <el-icon :size="44" style="color: #8b5cf6">
32           <PictureFilled
33         /></el-icon>
34       </template>
35     </el-empty>
36   </div>
37   <div v-else class="retrieval-grid">
38     <el-card
39       v-for="item in retrievalResults"
40       :key="item.id"
41       shadow="hover"
42       class="retrieval-card"
43       body-style="padding:0"
44     >
45       
6  <div class="retrieval-info">
7      <div class="retrieval-title">{{ item.title }}</div>
8      <div class="retrieval-sim">
9          <el-progress
10             :percentage="item.sim"
11             :color="
12                 item.sim > 80
13                     ? '#22c55e'
14                 : item.sim > 60
15                     ? '#f59e0b'
16                 : '#ef4444'
17             "
18             :stroke-width="6"
19         />
20         <span style="font-size: 11px; color: #909399"
21             >相似度 {{ item.sim }}%</span>
22     >
23 </div>
24 </div>
25 </el-card>
26 </div>
27 </div>
28 </el-tab-pane>
29
30 <!-- ===== Tab 4: Multimodal Chat ===== -->
31 <el-tab-pane label="多模态对话" name="chat">
32     <div style="display: flex; flex-direction: column; height: 100%">
33         <div class="thread" style="flex: 1">
34             <div v-if="mmChatMessages.length === 0" class="group-empty">
35                 <el-empty
36                     description="上传图片/音频/视频并提问，体验真正的多模态对话。"
37                     :image-size="96"
38                 >
39                     <template #image>
40                         <el-icon :size="44" style="color: #8b5cf6"
41                             ><ChatDotRound
42                             /></el-icon>
43                     </template>
44                 </el-empty>
45             </div>
46             <template v-for="msg in mmChatMessages" :key="msg.id">
47                 <div
48                     class="chat-bubble"
49                     :class="msg.role === 'user' ? 'is-user' : ''"
50                 >

```

```

1      <el-avatar
2        :size="36"
3        :icon="msg.role === 'user' ? User : Cpu"
4        :style="{
5          backgroundColor:
6            msg.role === 'user' ? '#1677ff' : '#8b5cf6',
7          flexShrink: 0,
8        }"
9      />
10     <div class="bubble-body">
11       <div class="bubble-name">
12         {{ msg.role === "user" ? "你" : "多模态助手" }}
13       </div>
14       <!-- User media attachments -->
15       <div
16         v-if="msg.role === 'user' && msg.image"
17         style="margin-bottom: 8px"
18       >
19         
29       </div>
30       <div
31         v-if="msg.role === 'user' && msg.audio"
32         style="margin-bottom: 8px"
33       >
34         <audio
35           :src="msg.audio"
36           controls
37           style="max-width: 260px; height: 36px"
38         />
39       </div>
40       <div
41         v-if="msg.role === 'user' && msg.video"
42         style="margin-bottom: 8px"
43       >
44         <video
45           :src="msg.video"
46           controls
47           style="
48             max-width: 260px;
49             max-height: 160px;
50             border-radius: 8px;

```

```

1      "
2      />
3    </div>
4    <div
5      v-if="msg.role === 'assistant'"
6      class="markdown-content"
7      v-html="
8        renderMarkdown(
9          msg.content ||
10            (isMmChatSubmitting ? '正在分析...' : ''),
11          )
12      "
13    ></div>
14    <p v-else>{{ msg.content }}</p>
15  </div>
16 </div>
17 </template>
18 </div>
19 <div class="composer">
20   <el-card class="composer-card" shadow="never">
21     <!-- Image previews (multi-image support) -->
22     <div
23       v-if="mmChatImages.length > 0"
24       style="
25         margin-bottom: 8px;
26         display: flex;
27         align-items: center;
28         gap: 6px;
29         flex-wrap: wrap;
30       "
31     >
32       <div
33         v-for="(img, idx) in mmChatImages"
34         :key="idx"
35         style="
36           display: flex;
37           align-items: center;
38           gap: 4px;
39           background: #fff5f9;
40           border-radius: 6px;
41           padding: 4px 8px;
42         "
43       >
44         
48         <span
49           style="
50             font-size: 11px;

```

```

1      color: #606266;
2      max-width: 100px;
3      overflow: hidden;
4      text-overflow: ellipsis;
5      white-space: nowrap;
6      "
7      >{{ img.name }}</span
8      >
9      <el-button
10     size="small"
11     text
12     type="danger"
13     @click="removeMmMedia(idx)"
14     style="padding: 0 4px"
15     > </el-button
16     >
17 </div>
18 </div>
19 <!-- Audio preview -->
20 <div
21   v-if="mmChatAudioBase64 && mmChatMediaType === 'audio'"
22   style="
23     margin-bottom: 8px;
24     display: flex;
25     align-items: center;
26     gap: 8px;
27     "
28   >
29   <el-tag type="success" size="small"> 音频</el-tag>
30   <span style="font-size: 12px; color: #909399">{{
31     mmChatAudioName
32   }}</span>
33   <el-button
34     size="small"
35     text
36     type="danger"
37     @click="removeMmMedia()"
38     >移除</el-button
39   >
40 </div>
41 <!-- Video preview -->
42 <div
43   v-if="mmChatVideoUrl && mmChatMediaType === 'video'"
44   style="
45     margin-bottom: 8px;
46     display: flex;
47     align-items: center;
48     gap: 8px;
49     "
50   >

```



```

1      <el-tag type="warning" size="small">  视频</el-tag>
2      <span style="font-size: 12px; color: #909399">{{
3          mmChatVideoName
4      }}</span>
5      <el-button
6          size="small"
7          text
8          type="danger"
9          @click="removeMmMedia()"
10         >移除</el-button
11     >
12 </div>
13 <div style="display: flex; gap: 8px">
14     <el-input
15         v-model="mmChatPrompt"
16         type="textarea"
17         :rows="2"
18         resize="vertical"
19         placeholder="输入问题，支持图片、音频、视频多模态对话..."
20         @keydown.enter.exact.prevent="submitMmChat()"
21         @compositionstart="isComposing = true"
22         @compositionend="isComposing = false"
23         style="flex: 1"
24         :disabled="isMmChatSubmitting"
25     />
26 </div>
27 <div class="composer-bar">
28     <div class="composer-meta" style="display: flex; gap: 6px">
29         <el-button size="small" @click="mmChatFileRef?.click()"
30             >  图片</el-button
31         >
32         <input
33             :ref="
34                 (el: any) => {
35                     mmChatFileRef = el;
36                 }
37             "
38             type="file"
39             accept="image/*"
40             style="display: none"
41             @change="handleMmImageUpload"
42         />
43         <el-button size="small" @click="mmChatAudioRef?.click()"
44             >  音频</el-button
45         >
46         <input
47             :ref="
48                 (el: any) => {
49                     mmChatAudioRef = el;
50                 }

```

```

1      "
2      type="file"
3      accept="audio/*"
4      style="display: none"
5      @change="handleMmAudioUpload"
6    />
7    <el-button size="small" @click="mmChatVideoRef?.click()"
8      > 视频</el-button
9    >
10   <input
11     :ref="
12       (el: any) => {
13         mmChatVideoRef = el;
14       }
15     "
16     type="file"
17     accept="video/*"
18     style="display: none"
19     @change="handleMmVideoUpload"
20   />
21 </div>
22 <div class="composer-actions">
23   <el-button :icon="Delete" @click="clearMmChat()"
24     >清空</el-button
25   >
26   <el-button
27     v-if="isMmChatSubmitting"
28     type="danger"
29     :icon="SwitchButton"
30     @click="stopMmChat()"
31     >停止</el-button
32   >
33   <el-button
34     v-else
35     type="primary"
36     :icon="Promotion"
37     @click="submitMmChat()"
38     :disabled="!mmChatPrompt.trim() || isMmChatSubmitting"
39     >{{ "发送" }}</el-button
40   >
41 </div>
42 </div>
43 </el-card>
44 </div>
45 </div>
46 </el-tab-pane>
47 </el-tabs>
48 </div>
49 </template>
50 // ===== frontend/src/components/app/pages/ConsoleModule.vue =====

```

```

1 <script setup lang="ts">
2 // @ts-nocheck
3 import { computed as vueComputed, isRef, unref } from 'vue';
4 import type { PageMode } from '../././types';
5
6 const props = defineProps<{ app: Record<string, any> }>();
7 const app = props.app;
8 const bind = (target: Record<string, any>, key: string) => vueComputed({
9   get: () => unref(target[key]),
10  set: (value) => {
11    if (isRef(target[key])) target[key].value = value;
12    else target[key] = value;
13  },
14 });
15
16 const SwitchButton = bind(app, 'SwitchButton');
17 const formatTime = bind(app, 'formatTime');
18 const isAuthDialogOpen = bind(app, 'isAuthDialogOpen');
19 const authUser = bind(app, 'authUser');
20 const rechargeOrders = bind(app, 'rechargeOrders');
21 const rechargeOrdersLoading = bind(app, 'rechargeOrdersLoading');
22 const billingLedger = bind(app, 'billingLedger');
23 const status = bind(app, 'status');
24 const isAuthLoaded = bind(app, 'isAuthLoaded');
25 const consoleDailyUsage = bind(app, 'consoleDailyUsage');
26 const consoleChartDays = bind(app, 'consoleChartDays');
27 const isAuthenticated = bind(app, 'isAuthenticated');
28 const authCreditsText = bind(app, 'authCreditsText');
29 const authSpentText = bind(app, 'authSpentText');
30 const handleUserMenu = bind(app, 'handleUserMenu');
31 const logout = bind(app, 'logout');
32 const openRechargeDialog = bind(app, 'openRechargeDialog');
33 const loadRechargeOrders = bind(app, 'loadRechargeOrders');
34 const consolePricingRows = bind(app, 'consolePricingRows');
35 const loadConsoleDailyUsage = bind(app, 'loadConsoleDailyUsage');
36 </script>
37
38 <template>
39   <div class="page-header">
40     <div class="header-left"><strong>控制台</strong></div>
41     <div class="header-right">
42       <template v-if="isAuthLoaded">
43         <el-button v-if="!isAuthenticated" type="primary" plain @click="isAuthDialogOpen = true">登录 / 注册</el-button>
44         <el-dropdown v-else trigger="click" @command="handleUserMenu">
45           <el-tag type="success" style="cursor:pointer">{{ authUser?.username }} </el-tag>
46           <template #dropdown>
47             <el-dropdown-menu>
48               <el-dropdown-item command="logout"><el-icon><SwitchButton /></el-icon> 退出登录</el-dropdown-item>
49             </el-dropdown-menu>
50           </template>

```

```

1      </el-dropdown>
2    </template>
3  </div>
4 </div>
5
6  <div class="console-page">
7    <el-alert v-if="isAuthLoaded && !isAuthenticated" type="warning" :closable="false" show-icon title="请先登录以查看控制台" style="margin-bottom:
8  16px" />
9    <template v-else-if="isAuthenticated">
10     <el-card shadow="never" class="console-section">
11       <template #header><strong>账户信息</strong></template>
12       <div class="console-stats-grid">
13         <div class="console-stat-card"><div class="console-stat-label">用户名</div><div class="console-stat-value">{{ authUser?.username }}</div></div>
14       <div class="console-stat-card highlight"><div class="console-stat-label">余额</div><div class="console-stat-value">¥ {{ authCreditsText }}</div></div>
15       <div class="console-stat-card"><div class="console-stat-label">累计消费</div><div class="console-stat-value">¥ {{ authSpentText }}</div></div>
16       <div class="console-stat-card"><div class="console-stat-label">注册时间</div><div class="console-stat-value">{{ formatTime(authUser?.createdAt) }}</div></div>
17     </div>
18     <el-divider />
19     <div style="display: flex; align-items: center; gap: 12px; flex-wrap: wrap">
20       <span style="font-size: 14px; color: #374151">在线充值: </span>
21       <el-button type="primary" @click="openRechargeDialog()">支付宝当面付</el-button>
22     </div>
23   </el-card>
24
25   <el-card shadow="never" class="console-section">
26     <template #header>
27       <div style="display: flex; align-items: center; justify-content: space-between">
28         <strong>充值记录</strong>
29         <el-button text size="small" @click="loadRechargeOrders()" :loading="rechargeOrdersLoading">刷新</el-button>
30       </div>
31     </template>
32     <el-table v-if="rechargeOrders.length > 0" :data="rechargeOrders" size="small" max-height="240">
33       <el-table-column prop="amount" label="金额" width="80">
34         <template #default="{ row }">¥ {{ row.amount }}</template>
35       </el-table-column>
36       <el-table-column prop="status" label="状态" width="90">
37         <template #default="{ row }">
38           <el-tag type="row.status === 'paid' ? 'success' : row.status === 'pending' ? 'warning' : 'info'" size="small">
39             {{ row.status === 'paid' ? '已支付' : row.status === 'pending' ? '待支付' : row.status }}
40           </el-tag>
41         </template>
42       </el-table-column>
43       <el-table-column prop="alipayTradeNo" label="支付宝交易号" min-width="160">
44         <template #default="{ row }">{{ row.alipayTradeNo || '-' }}</template>
45       </el-table-column>
46       <el-table-column prop="createdAt" label="创建时间" min-width="140">
47         <template #default="{ row }">{{ formatTime(row.createdAt) }}</template>

```

```

1      </el-table-column>
2      <el-table-column prop="paidAt" label="支付时间" min-width="140">
3          <template #default="{ row }">{{ row.paidAt ? formatTime(row.paidAt) : '-' }}</template>
4      </el-table-column>
5  </el-table>
6  <el-empty v-else description="暂无充值记录" :image-size="48" style="padding:20px 0" />
7  </el-card>
8
9  <el-card shadow="never" class="console-section">
10     <template #header>
11         <div style="display:flex;align-items:center;justify-content:space-between">
12             <strong>用量趋势</strong>
13             <el-segmented
14                 :model-value="String(consoleChartDays)"
15                 @change="(val: string | number) => { consoleChartDays = Number(val); loadConsoleDailyUsage(); }"
16                 :options="[
17                     { label: '7天', value: '7' },
18                     { label: '30天', value: '30' },
19                     { label: '90天', value: '90' },
20                 ]"
21                 size="small"
22             />
23         </div>
24     </template>
25     <div v-if="consoleDailyUsage.length > 0" id="console-chart" style="width:100%;height:260px"></div>
26     <el-empty v-else description="暂无用量数据" :image-size="48" style="padding:30px 0" />
27 </el-card>
28
29 <el-card shadow="never" class="console-section">
30     <template #header>
31         <div style="display:flex;align-items:center;justify-content:space-between">
32             <strong>计费规则</strong>
33             <span style="font-size:11px;color:#9ca3af">元 / 千 token</span>
34         </div>
35     </template>
36     <el-table :data="consolePricingRows" stripe style="width:100%">
37         <el-table-column label="档位" width="120">
38             <template #default="{ row }">
39                 <el-tag :type="row.tagType" size="small">{{ row.label }}</el-tag>
40             </template>
41         </el-table-column>
42         <el-table-column label="模型举例" min-width="250">
43             <template #default="{ row }">{{ row.sampleModels }}</template>
44         </el-table-column>
45         <el-table-column label="输入价格" width="130" align="center">
46             <template #default="{ row }"> {{ row.promptPrice.toFixed(4) }}</template>
47         </el-table-column>
48         <el-table-column label="输出价格" width="130" align="center">
49             <template #default="{ row }"> {{ row.completionPrice.toFixed(4) }}</template>
50         </el-table-column>

```

```

1      <template #empty><el-empty description="暂无计费规则" :image-size="48" /></template>
2
3    </el-card>
4
5    <el-card shadow="never" class="console-section">
6      <template #header><strong>用量明细</strong></template>
7      <el-table :data="billingLedger" stripe style="width:100%">
8        <template #empty><el-empty description="暂无用量记录，使用 API 后将自动记录" :image-size="48" /></template>
9        <el-table-column prop="model" label="模型" min-width="220" />
10       <el-table-column prop="requestType" label="请求类型" min-width="120" />
11       <el-table-column prop="promptTokens" label="输入Token" min-width="130" />
12       <el-table-column prop="completionTokens" label="输出Token" min-width="130" />
13       <el-table-column prop="cost" label="费用" min-width="110" />
14       <el-table-column label="时间" min-width="200">
15         <template #default="{ row }">{{ formatTime(row.createdAt) }}</template>
16       </el-table-column>
17     </el-table>
18   </el-card>
19 </template>
20 </div>
21 </template>
22 // ===== frontend/src/components/app/pages/CollabModule.vue =====
23 <script setup lang="ts">
24 // @ts-nocheck
25 import { computed as vueComputed, isRef, unref } from 'vue';
26 import type { PageMode } from '../types';
27
28 const props = defineProps<{ app: Record<string, any> }>();
29 const app = props.app;
30 const bind = (target: Record<string, any>, key: string) => vueComputed({
31   get: () => unref(target[key]),
32   set: (value) => {
33     if (isRef(target[key])) target[key].value = value;
34     else target[key] = value;
35   },
36 });
37
38 const DataAnalysis = bind(app, 'DataAnalysis');
39 const Delete = bind(app, 'Delete');
40 const Promotion = bind(app, 'Promotion');
41 const Star = bind(app, 'Star');
42 const SwitchButton = bind(app, 'SwitchButton');
43 const TrendCharts = bind(app, 'TrendCharts');
44 const renderMarkdown = bind(app, 'renderMarkdown');
45 const isAuthDialogOpen = bind(app, 'isAuthDialogOpen');
46 const authUser = bind(app, 'authUser');
47 const models = bind(app, 'models');
48 const status = bind(app, 'status');
49 const isAuthLoaded = bind(app, 'isAuthLoaded');
50 const isComposing = bind(app, 'isComposing');

```

```

1 const collabMode = bind(app, 'collabMode');
2 const collabPrompt = bind(app, 'collabPrompt');
3 const collabRunning = bind(app, 'collabRunning');
4 const collabSelectedModels = bind(app, 'collabSelectedModels');
5 const collabPanels = bind(app, 'collabPanels');
6 const collabSummary = bind(app, 'collabSummary');
7 const collabSummaryStatus = bind(app, 'collabSummaryStatus');
8 const getCollabModelLogo = bind(app, 'getCollabModelLogo');
9 const isAuthenticated = bind(app, 'isAuthenticated');
10 const handleUserMenu = bind(app, 'handleUserMenu');
11 const logout = bind(app, 'logout');
12 const startCollab = bind(app, 'startCollab');
13 const clearCollab = bind(app, 'clearCollab');
14 const stopCollab = bind(app, 'stopCollab');
15 </script>
16
17 <template>
18   <div class="page-header">
19     <div class="header-left" style="gap:8px">
20       <strong>协同推理</strong>
21       <el-radio-group v-model="collabMode" size="small" :disabled="collabRunning">
22         <el-radio-button value="debate">并行辩论</el-radio-button>
23         <el-radio-button value="review">同行评审</el-radio-button>
24         <el-radio-button value="divide">分工协作</el-radio-button>
25       </el-radio-group>
26     </div>
27     <div class="header-right">
28       <template v-if="isAuthLoaded">
29         <el-button v-if="!isAuthenticated" type="primary" plain @click="isAuthDialogOpen = true">登录 / 注册</el-button>
30         <el-dropdown v-else trigger="click" @command="handleUserMenu">
31           <el-tag type="success" style="cursor:pointer">{{ authUser?.username }} </el-tag>
32           <template #dropdown>
33             <el-dropdown-menu>
34               <el-dropdown-item command="console"><el-icon><DataAnalysis /></el-icon> 控制台</el-dropdown-item>
35               <el-dropdown-item command="logout" divided><el-icon><SwitchButton /></el-icon> 退出登录</el-dropdown-item>
36             </el-dropdown-menu>
37           </template>
38         </el-dropdown>
39       </template>
40     </div>
41   </div>
42
43   <div class="thread" style="flex:1">
44     <div v-if="collabPanels.length === 0 && !collabRunning" class="group-empty">
45       <el-empty description="输入问题，多个AI模型将协同推理并返回最优结果。" :image-size="96">
46         <template #image><el-icon :size="44" style="color:#8b5cf6"><TrendCharts /></el-icon></template>
47       </el-empty>
48     </div>
49
50     <div v-else style="display:flex;flex-direction:column;gap:16px">

```

```

1      <!-- Model panels with logos -->
2      <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(300px,1fr));gap:12px">
3          <el-card v-for="p in collabPanels" :key="p.modelId" shadow="hover" :style="{ borderLeft: `3px solid ${p.status === 'done' ? '#22c55e' : p.status ===
4 'streaming' ? '#3b82f6' : '#d1d5db'}`, transition: 'border-color .3s' }">
5              <template #header>
6                  <div style="display:flex;align-items:center;justify-content:space-between">
7                      <div style="display:flex;align-items:center;gap:8px">
8                          <el-avatar :size="24" :style="{ backgroundColor: getCollabModelLogo(p.modelId).color, fontSize:'11px', fontWeight:700 }">{{ getCollabModelLogo(p.
9 modelId).initial }}</el-avatar>
10                             <span style="font-weight:600;font-size:13px">{{ p.modelName }}</span>
11                         </div>
12                     <el-tag :type="p.status === 'done' ? 'success' : p.status === 'streaming' ? 'warning' : 'info'" size="small">{{ p.status === 'done' ? ' 完成' : p.status
13 === 'streaming' ? ' 生成中' : '○ 等待' }}</el-tag>
14                 </div>
15             </template>
16             <div class="markdown-content" v-html="renderMarkdown(p.content || (p.status === 'streaming' ? '  : '等待回复'))" style="font-size:13px;line-height:1
17 .7;min-height:20px"></div>
18         </el-card>
19     </div>
20
21     <!-- Continue / Refine button -->
22     <div v-if="collabPanels.length > 0 && collabSummaryStatus === 'done' && !collabRunning" style="display:flex;gap:8px;justify-content:center">
23         <el-button size="small" @click="startCollab('请综合以上结果，给出最终建议')"> 继续讨论</el-button>
24         <el-button size="small" @click="startCollab('请指出以上回答中可能存在的问题')"> 质疑分析</el-button>
25         <el-button size="small" @click="startCollab('请用一句话总结以上讨论')"> 一句话总结</el-button>
26     </div>
27
28     <!-- Summary panel -->
29     <el-card v-if="collabSummaryStatus !== 'idle'" shadow="hover" style="border-left:3px solid #8b5cf6">
30         <template #header>
31             <div style="display:flex;align-items:center;gap:8px">
32                 <el-icon color="#8b5cf6"><Star /></el-icon>
33                 <strong style="font-size:13px">汇总结果</strong>
34             <el-tag v-if="collabSummaryStatus === 'streaming'" type="warning" size="small">生成中</el-tag>
35             <el-tag v-else type="success" size="small">完成</el-tag>
36         </div>
37     </template>
38     <div class="markdown-content" v-html="renderMarkdown(collabSummary || '正在汇总...')" style="font-size:14px;line-height:1.7"></div>
39 </el-card>
40 </div>
41 </div>
42
43 <div class="composer">
44     <el-card class="composer-card" shadow="never">
45         <!-- Model selector -->
46         <div style="margin-bottom:6px;display:flex;gap:6px;flex-wrap:wrap;align-items:center">
47             <span style="font-size:11px;color:#909399;flex-shrink:0">模型:</span>
48             <el-select v-model="collabSelectedModels" multiple filterable placeholder="自动选择" size="small" style="flex:1;min-width:200px" :
49 disabled="collabRunning" collapse-tags collapse-tags-tooltip>
50                 <el-option v-for="m in models.filter(x => !x.id.includes('vl') && !x.id.includes('tts') && !x.id.includes('voice'))" :key="m.id" :label="m.id" :value="m.id" />

```



```

1      </el-select>
2      </div>
3      <el-input v-model="collabPrompt" type="textarea" :rows="2" resize="vertical" placeholder="输入问题，多个模型将协同推理..." :
4      disabled="collabRunning" @keydown.enter.exact.prevent="startCollab()" @compositionstart="isComposing = true" @compositionend="isComposing = false" /
5      >
6      <div class="composer-bar">
7      <div class="composer-meta">
8      <span class="meta-text">{{ collabSelectedModels.length > 0 ? `已选${collabSelectedModels.length}个模型` : '自动选择最佳模型' }}</span>
9      </div>
10     <div class="composer-actions">
11     <el-button icon="Delete" @click="clearCollab()" size="small">清空</el-button>
12     <el-button v-if="collabRunning" type="danger" @click="stopCollab()" size="small">停止</el-button>
13     <el-button v-else type="primary" icon="Promotion" @click="startCollab()" :disabled="!collabPrompt.trim()" size="small">发送</el-button>
14     </div>
15     </div>
16   </el-card>
17 </div>
18 </template>
19 // ===== frontend/src/types/agent.ts =====
20 // Agent相关类型
21 export interface AgentForm {
22   id: string;
23   name: string;
24   description: string;
25   model: string;
26   systemPrompt: string;
27   temperature: number;
28   maxTokens: number;
29   memoryEnabled: boolean;
30   toolIds: string[];
31   knowledgeBaseIds: string[];
32   skillIds: string[];
33   workflowIds: string[];
34   subAgentIds: string[];
35   status: 'active' | 'archived';
36   published: boolean;
37   apiEnabled: boolean;
38   publicSlug: string;
39 }
40
41 // DAG节点类型
42 export type DagNodeType =
43   | 'start' | 'user_input' | 'file_input' | 'form_input'
44   | 'output' | 'structured_output' | 'generate_file' | 'end'
45   | 'intent_detection' | 'parameter_extract' | 'info_extract'
46   | 'content_classify' | 'condition' | 'if_else' | 'multi_branch' | 'question_classifier' | 'confidence_check'
47   | 'knowledge_search' | 'context_read' | 'memory_read' | 'memory_write'
48   | 'doc_parse' | 'citation整理'
49   | 'tool_call' | 'skill_call' | 'agent_call' | 'http_request'
50   | 'db_query' | 'code执行' | 'code' | 'webhook'

```

```

1  | 'llm_generate' | 'prompt_builder' | 'template_transform' | 'variable_assigner' | 'result_summary' | 'result_rewrite'
2  | 'format_output' | 'json_parse' | 'multi_result_merge'
3  | 'human_confirm' | 'permission_check' | 'sensitive_confirm'
4  | 'retry' | 'error_handle' | 'fallback' | 'wait';
5
6 export interface DagNodeItem {
7   type: DagNodeType;
8   label: string;
9   description: string;
10  icon: string;
11  category: string;
12  inputs: string[];
13  outputs: string[];
14  configSchema: Record<string, any>;
15 }
16
17 export interface DagNodeCategory {
18   id: string;
19   label: string;
20   icon: string;
21   nodes: DagNodeItem[];
22 }
23
24 export interface DagTemplate {
25   id: string;
26   name: string;
27   description: string;
28   icon: string;
29   category: string;
30   nodes: any[];
31   edges: any[];
32 }
33
34 // Tool相关类型
35 export interface CustomTool {
36   id: string;
37   name: string;
38   displayName: string;
39   description: string;
40   icon: string;
41   category: string;
42   type: 'builtin' | 'custom';
43   runtime: 'python' | 'javascript' | 'http' | 'webhook' | 'db';
44   status: 'enabled' | 'disabled';
45   riskLevel: 'low' | 'medium' | 'high';
46   version: string;
47   inputSchema: Record<string, any>;
48   outputSchema: Record<string, any>;
49   permissions: {
50     network: boolean;

```

```
1  database: boolean;
2  fileRead: boolean;
3  fileWrite: boolean;
4  externalRequest: boolean;
5  };
6  code?: string;
7  entry?: string;
8  exampleInput?: string;
9  exampleOutput?: string;
10 timeout: number;
11 retries: number;
12 createdAt: string;
13 updatedAt: string;
14 callCount: number;
15 lastCallAt?: string;
16 }
17
18 // Skill相关类型
19 export interface SkillDefinition {
20   id: string;
21   name: string;
22   description: string;
23   content: string;
24   category: string;
25   icon: string;
26   riskLevel: 'low' | 'medium' | 'high';
27   source: 'builtin' | 'custom';
28   version: number;
29   enabled: boolean;
30   bindingCount: number;
31   inputSchema?: Record<string, any>;
32   outputSchema?: Record<string, any>;
33   exampleInput?: string;
34   exampleOutput?: string;
35   permissions?: Record<string, boolean>;
36   boundAgents?: Array<{ id: string; name: string }>;
37 }
38
39 // Knowledge相关类型
40 export interface KnowledgeBase {
41   id: string;
42   name: string;
43   description: string;
44   provider?: 'native' | 'ragflow';
45   externalId?: string | null;
46   documentCount: number;
47   chunkCount: number;
48   createdAt: string;
49   updatedAt: string;
50 }
```

```

1
2 export interface KnowledgeDocument {
3   id: string;
4   kbId: string;
5   title: string;
6   content: string;
7   chunkCount: number;
8   createdAt: string;
9 }
10 // ===== frontend/src/components/app/pages/GroupModule.vue =====
11 <script setup lang="ts">
12 // @ts-nocheck
13 import { computed as vueComputed, isRef, unref } from 'vue';
14 import type { PageMode } from '../types';
15
16 const props = defineProps<{ app: Record<string, any> }>();
17 const app = props.app;
18 const bind = (target: Record<string, any>, key: string) => vueComputed({
19   get: () => unref(target[key]),
20   set: (value) => {
21     if (isRef(target[key])) target[key].value = value;
22     else target[key] = value;
23   },
24 });
25
26 const DataAnalysis = bind(app, 'DataAnalysis');
27 const Delete = bind(app, 'Delete');
28 const Promotion = bind(app, 'Promotion');
29 const Refresh = bind(app, 'Refresh');
30 const SwitchButton = bind(app, 'SwitchButton');
31 const User = bind(app, 'User');
32 const UserFilled = bind(app, 'UserFilled');
33 const getModelLogo = bind(app, 'getModelLogo');
34 const renderMarkdown = bind(app, 'renderMarkdown');
35 const isAuthDialogOpen = bind(app, 'isAuthDialogOpen');
36 const authUser = bind(app, 'authUser');
37 const userInvitationCode = bind(app, 'userInvitationCode');
38 const models = bind(app, 'models');
39 const isLoadingModels = bind(app, 'isLoadingModels');
40 const status = bind(app, 'status');
41 const isAuthLoaded = bind(app, 'isAuthLoaded');
42 const isComposing = bind(app, 'isComposing');
43 const groupPrompt = bind(app, 'groupPrompt');
44 const isGrouping = bind(app, 'isGrouping');
45 const groupMessages = bind(app, 'groupMessages');
46 const groupThreadRef = bind(app, 'groupThreadRef');
47 const isAuthenticated = bind(app, 'isAuthenticated');
48 const loadModels = bind(app, 'loadModels');
49 const handleUserMenu = bind(app, 'handleUserMenu');
50 const logout = bind(app, 'logout');

```

```

1 const startGroupChat = bind(app, 'startGroupChat');
2 const clearGroupChat = bind(app, 'clearGroupChat');
3 const stopGroupChat = bind(app, 'stopGroupChat');
4 const groupModelList = bind(app, 'groupModelList');
5 const copyToClipboard = bind(app, 'copyToClipboard');
6 </script>
7
8 <template>
9   <div class="page-header">
10     <div class="header-left">
11       <el-button :icon="Refresh" :loading="isLoadingModels" @click="loadModels()">刷新模型</el-button>
12     </div>
13     <div class="header-right">
14       <template v-if="isAuthLoaded">
15         <el-button v-if="!isAuthenticated" type="primary" plain @click="isAuthDialogOpen = true">登录 / 注册</el-button>
16         <el-dropdown v-else trigger="click" @command="handleUserMenu">
17           <el-tag type="success" style="cursor:pointer">{{ authUser?.username }} </el-tag>
18           <template #dropdown>
19             <el-dropdown-menu>
20               <el-dropdown-item command="console"><el-icon><DataAnalysis /></el-icon> 控制台</el-dropdown-item>
21               <el-dropdown-item v-if="userInvitationCode" disabled>
22                 <span style="color:#666;">邀请码: {{ userInvitationCode }}</span>
23                 <el-button size="small" style="margin-left:8px" @click.stop="copyToClipboard(userInvitationCode)">复制</el-button>
24               </el-dropdown-item>
25               <el-dropdown-item command="logout" divided><el-icon><SwitchButton /></el-icon> 退出登录</el-dropdown-item>
26             </el-dropdown-menu>
27           </template>
28         </el-dropdown>
29         <el-tag :type="isGrouping ? undefined : 'success'">{{ isGrouping ? '群聊进行中...' : `${groupModelList.length} 个参与模型` }}</el-tag>
30       </template>
31     </div>
32 </div>
33
34 <div ref="groupThreadRef" class="thread group-thread">
35   <div v-if="groupMessages.length === 0" class="group-empty">
36     <el-empty description="输入消息，AI 们将依次讨论回复。" :image-size="96">
37       <template #image>
38         <el-icon :size="44" style="color: #0ea5e9"><UserFilled /></el-icon>
39       </template>
40     </el-empty>
41   </div>
42
43   <template v-for="msg in groupMessages" :key="msg.id">
44     <div v-if="msg.role === 'user'" class="chat-bubble is-user">
45       <el-avatar :size="36" :icon="User" style="background-color:#1677ff;flex-shrink:0" />
46       <div class="bubble-body">
47         <div class="bubble-name">你</div>
48         <p>{{ msg.content }}</p>
49       </div>
50     </div>

```

```

1
2   <div v-else class="chat-bubble">
3     <el-avatar
4       :size="36"
5       :src="getModelLogo(msg.model || '').url || undefined"
6       :style="{ backgroundColor: getModelLogo(msg.model || '').color, flexShrink: 0 }"
7     >
8       {{ getModelLogo(msg.model || '').name.charAt(0).toUpperCase() }}
9     </el-avatar>
10    <div class="bubble-body">
11      <div class="bubble-name">
12        {{ msg.model }}
13        <el-tag v-if="msg.status === 'streaming'" size="small" effect="dark" class="streaming-dot">生成中</el-tag>
14      </div>
15      <div
16        class="markdown-content group-content"
17        :class="{ 'is-streaming-update': msg.status === 'streaming' }"
18        v-html="renderMarkdown(msg.content || '正在生成回复...')"
19      ></div>
20    </div>
21  </div>
22 </template>
23 </div>
24
25 <div class="composer">
26   <el-card class="composer-card" shadow="never">
27     <el-input
28       v-model="groupPrompt"
29       type="textarea"
30       :rows="3"
31       resize="vertical"
32       placeholder="输入消息，所有模型将逐个回复"
33       :disabled="isGrouping"
34       @keydown.enter.exact.prevent="startGroupChat()"
35       @compositionstart="isComposing = true"
36       @compositionend="isComposing = false"
37     />
38     <div class="composer-bar">
39       <div class="composer-meta">
40         <el-text type="info" size="small">{{ models.length }} 个模型</el-text>
41       </div>
42       <div class="composer-actions">
43         <el-button :icon="Delete" @click="clearGroupChat()">清空</el-button>
44         <el-button v-if="isGrouping" type="danger" :icon="SwitchButton" @click="stopGroupChat()">停止</el-button>
45         <el-button v-else type="primary" :icon="Promotion" @click="startGroupChat()" :disabled="!groupPrompt.trim()">发送</el-button>
46       </div>
47     </div>
48   </el-card>
49 </div>
50 </template>

```

```

1 // ===== backend/src/agents/agents.service.spec.ts =====
2 jest.mock('@langchain/langgraph', () => ({
3   Annotation: { Root: jest.fn() },
4   START: '__start__',
5   END: '__end__',
6   StateGraph: jest.fn(),
7 }));
8
9 import { AgentRun, AgentsService } from './agents.service';
10 import { ToolDefinition } from '../tools/tools.service';
11
12 function makeTool(overrides: Partial<ToolDefinition> = {}): ToolDefinition {
13   return {
14     id: overrides.id ?? 'tool-1',
15     userId: overrides.userId ?? 'user-1',
16     name: overrides.name ?? 'calculator',
17     displayName: overrides.displayName ?? 'Calculator',
18     description: overrides.description ?? 'Calculate arithmetic expressions',
19     schema: overrides.schema ?? {
20       type: 'object',
21       properties: { expression: { type: 'string' } },
22       required: ['expression'],
23     },
24     outputSchema: overrides.outputSchema,
25     permissions: overrides.permissions ?? {},
26     implementationType: overrides.implementationType ?? 'builtin',
27     source: overrides.source ?? 'builtin',
28     category: overrides.category ?? 'utility',
29     runtime: overrides.runtime ?? 'javascript',
30     riskLevel: overrides.riskLevel ?? 'low',
31     code: overrides.code,
32     version: overrides.version ?? 1,
33     callCount: overrides.callCount ?? 0,
34     timeoutMs: overrides.timeoutMs ?? 30000,
35     retries: overrides.retries ?? 0,
36     enabled: overrides.enabled ?? true,
37     permissionLevel: overrides.permissionLevel ?? 'auto',
38   };
39 }
40
41 function makeService(overrides: {
42   completions?: unknown[];
43   invoke?: jest.Mock;
44   getAgentTools?: jest.Mock;
45 } = {}) {
46   const chatService = {
47     createCompletion: jest.fn(),
48   };
49   for (const completion of overrides.completions ?? []) {
50     chatService.createCompletion.mockResolvedValueOnce(completion);

```

```
1  }
2
3  const billingService = {
4    chargeForCompletion: jest.fn(async () => ({ credits: 99 })),
5    getBalance: jest.fn(async () => ({ credits: 99 })),
6  };
7  const toolsService = {
8    invoke: overrides.invoke ?? jest.fn(async () => ({
9      id: 'inv-1',
10     toolId: 'tool-1',
11     toolName: 'calculator',
12     input: { expression: '1+2' },
13     output: '3',
14     status: 'succeeded',
15     error: '',
16     latencyMs: 3,
17   })),
18   getAgentTools: overrides.getAgentTools ?? jest.fn(),
19   listForUser: jest.fn(),
20 };
21 const databaseService = {
22   now: jest.fn(() => '2026-06-09 09:00:00.000'),
23   connection: { prepare: jest.fn() },
24 };
25
26 const service = new AgentsService(
27   databaseService as never,
28   chatService as never,
29   billingService as never,
30   toolsService as never,
31   {} as never,
32   {} as never,
33   {} as never,
34   {} as never,
35 );
36 return { service, chatService, billingService, toolsService };
37 }
38
39 const agent = {
40   id: 'agent-1',
41   userId: 'user-1',
42   name: 'Harness Agent',
43   description: '',
44   model: 'qwen-plus',
45   systemPrompt: '',
46   temperature: 0.2,
47   maxTokens: 1024,
48   memoryEnabled: false,
49   toolIds: ['tool-1'],
50   knowledgeBaseIds: [],
```



```

1  skillIds: [],
2  workflowIds: [],
3  subAgentIds: [],
4  published: false,
5  apiEnabled: false,
6  publicSlug: "",
7  status: 'active' as const,
8  createdAt: "",
9  updatedAt: "",
10 runCount: 0,
11 };
12
13 function makeRun(overrides: Partial<AgentRun> = {}): AgentRun {
14   return {
15     id: overrides.id ?? 'run-1',
16     agentId: overrides.agentId ?? 'agent-1',
17     userId: overrides.userId ?? 'user-1',
18     status: overrides.status ?? 'succeeded',
19     conversationId: overrides.conversationId ?? 'conversation-1',
20     parentRunId: overrides.parentRunId ?? null,
21     input: overrides.input ?? '写一段论文摘要',
22     output: overrides.output ?? '这是一段完整的论文摘要，包含研究背景、方法、发现和结论，足以触发自动评测输出分。',
23     model: overrides.model ?? 'qwen-plus',
24     error: overrides.error ?? "",
25     promptTokens: overrides.promptTokens ?? 100,
26     completionTokens: overrides.completionTokens ?? 200,
27     totalTokens: overrides.totalTokens ?? 300,
28     latencyMs: overrides.latencyMs ?? 1200,
29     createdAt: overrides.createdAt ?? '2026-06-09 09:00:00.000',
30     completedAt: overrides.completedAt ?? '2026-06-09 09:00:01.200',
31     steps: overrides.steps ?? [{
32       id: 1,
33       runId: overrides.id ?? 'run-1',
34       stepType: 'llm_completion',
35       name: '最终回答',
36       status: 'succeeded',
37       input: overrides.input ?? '写一段论文摘要',
38       output: overrides.output ?? '这是一段完整的论文摘要，包含研究背景、方法、发现和结论，足以触发自动评测输出分。',
39       error: "",
40       startedAt: '2026-06-09 09:00:00.000',
41       endedAt: '2026-06-09 09:00:01.200',
42       latencyMs: 1200,
43       metadata: '{}',
44     }],
45   };
46 }
47
48 function completion(content: string, toolCalls: unknown[] = []) {
49   return {
50     id: `cml-${Math.random()}`,

```

```

1  object: 'chat.completion',
2  created: 0,
3  model: 'qwen-plus',
4  choices: [{
5    index: 0,
6    message: {
7      role: 'assistant',
8      content,
9      ...(toolCalls.length ? { tool_calls: toolCalls } : {}),
10   },
11   finish_reason: toolCalls.length ? 'tool_calls' : 'stop',
12  }],
13  usage: { prompt_tokens: 10, completion_tokens: 4, total_tokens: 14 },
14 };
15 }
16
17 async function completeStep(
18   stepId: number,
19   status: 'succeeded' | 'failed',
20   output: string,
21   error = "",
22   metadata: Record<string, unknown> = {},
23 ) {
24   return {
25     id: stepId ?? 0,
26     runId: 'run-1',
27     stepType: 'llm_completion',
28     name: '完成步骤',
29     status,
30     input: "",
31     output,
32     error,
33     startedAt: '2026-06-09 09:00:00.000',
34     endedAt: '2026-06-09 09:00:00.100',
35     latencyMs: 100,
36     metadata: JSON.stringify(metadata),
37   };
38 }
39
40 describe('AgentsService function calling harness', () => {
41   it('converts bound tools to OpenAI function definitions', () => {
42     const { service } = makeService();
43     const converted = (service as any).toolDefinitionToChatTool(makeTool());
44     expect(converted).toEqual({
45       type: 'function',
46       function: {
47         name: 'calculator',
48         description: 'Calculate arithmetic expressions',
49         parameters: {
50           type: 'object',

```

```

1    properties: { expression: { type: 'string' } },
2    required: ['expression'],
3  },
4  },
5  });
6
7  const emptySchema = (service as any).toolDefinitionToChatTool(makeTool({ schema: {} }));
8  expect(emptySchema.function.parameters).toEqual({ type: 'object', properties: {} });
9  });
10
11 it('executes a model tool_call and feeds the tool result back into the next LLM turn', async () => {
12   const toolCall = {
13     id: 'call-1',
14     type: 'function',
15     function: { name: 'calculator', arguments: '{"expression": "1+2"}' },
16   };
17   const { service, chatService, toolsService, billingService } = makeService({
18     completions: [completion("", [toolCall]), completion('结果是 3')],
19   });
20   const steps: unknown[] = [];
21
22   const output = await (service as any).runToolUseHarness(
23     'user-1',
24     agent,
25     { input: '计算 1+2' },
26     'run-1',
27     { model: 'qwen-plus', messages: [{ role: 'user', content: '计算 1+2' }] },
28     [makeTool()],
29     [],
30     3,
31     async (step: unknown) => { steps.push(step); return step; },
32     completeStep,
33     { prompt_tokens: 0, completion_tokens: 0, total_tokens: 0 },
34   );
35
36   expect(output).toBe('结果是 3');
37   expect(toolsService.invoke).toHaveBeenCalledWith('user-1', 'tool-1', { expression: '1+2' }, { agentId: 'agent-1', runId: 'run-1' });
38   expect(chatService.createCompletion).toHaveBeenCalledTimes(2);
39   expect((chatService.createCompletion.mock.calls[1][0].messages)).toEqual(expect.arrayContaining([
40     expect.objectContaining({ role: 'assistant', tool_calls: [toolCall] }),
41     expect.objectContaining({ role: 'tool', tool_call_id: 'call-1', content: '3', name: 'calculator' }),
42   ]));
43   expect(steps).toEqual(expect.arrayContaining([
44     expect.objectContaining({ stepType: 'tool_call', status: 'succeeded' }),
45     expect.objectContaining({ stepType: 'observation', status: 'succeeded' }),
46   ]));
47   expect(billingService.chargeForCompletion).toHaveBeenCalledTimes(2);
48 });
49
50 it('executes multiple tool_calls in return order', async () => {

```

```

1  const calls = [
2    { id: 'call-1', type: 'function', function: { name: 'calculator', arguments: '{"expression":"1+2"}' } },
3    { id: 'call-2', type: 'function', function: { name: 'current_time', arguments: '{"timezone":"Asia/Shanghai"}' } },
4  ];
5  const invoke = jest.fn()
6    .mockResolvedValueOnce({ output: '3', status: 'succeeded', error: "", id: '1', toolId: 'tool-1', toolName: 'calculator', input: {}, latencyMs: 1 })
7    .mockResolvedValueOnce({ output: '星期二', status: 'succeeded', error: "", id: '2', toolId: 'tool-2', toolName: 'current_time', input: {}, latencyMs: 1 });
8  const { service } = makeService({ completions: [completion("", calls), completion("完成")], invoke });
9
10 await (service as any).runToolUseHarness(
11   'user-1',
12   agent,
13   { input: '计算并告诉我时间' },
14   'run-1',
15   { model: 'qwen-plus', messages: [{ role: 'user', content: '计算并告诉我时间' }] },
16   [makeTool(), makeTool({ id: 'tool-2', name: 'current_time', displayName: 'Current time', schema: { type: 'object', properties: { timezone: { type: 'string' } } } })],
17   [],
18   3,
19   async (step: unknown) => step,
20   completeStep,
21   { prompt_tokens: 0, completion_tokens: 0, total_tokens: 0 },
22 );
23
24 expect(invoke.mock.calls.map((call) => call[1])).toEqual(['tool-1', 'tool-2']);
25 });
26
27 it('executes delegate_agent calls and feeds the child result back into the next LLM turn', async () => {
28   const delegateCall = {
29     id: 'call-delegate',
30     type: 'function',
31     function: { name: 'delegate_agent', arguments: '{"agentId":"agent-2","task":"核查事实","reason":"需要专家复核"}' },
32   };
33   const childAgent = { ...agent, id: 'agent-2', name: 'Fact Checker', subAgentIds: [] };
34   const { service, chatService } = makeService({
35     completions: [completion("", [delegateCall]), completion("已经综合子 Agent 结果")],
36   });
37   (service as any).delegateUserAgent = jest.fn(async () => ({
38     observation: 'Fact Checker (succeeded) runId=child-run\n事实无误',
39     run: makeRun({ id: 'child-run', agentId: 'agent-2', output: '事实无误' }),
40   }));
41
42   const output = await (service as any).runToolUseHarness(
43     'user-1',
44     { ...agent, subAgentIds: ['agent-2'] },
45     { input: '请复核事实' },
46     'run-1',
47     { model: 'qwen-plus', messages: [{ role: 'user', content: '请复核事实' }] },
48     [],
49     [childAgent],
50     3,

```

```

1  async (step: unknown) => step,
2  completeStep,
3  { prompt_tokens: 0, completion_tokens: 0, total_tokens: 0 },
4  { delegationDepth: 0 },
5  );
6
7  expect(output).toBe('已经综合子 Agent 结果');
8  expect((service as any).delegateUserAgent).toHaveBeenCalledWith(
9    'user-1',
10   expect.objectContaining({ id: 'agent-1' }),
11   expect.objectContaining({ input: '请复核事实' }),
12   expect.objectContaining({ action: 'delegate', agentId: 'agent-2', input: '核查事实' }),
13   'run-1',
14   expect.any(Object),
15   expect.any(Function),
16   expect.any(Function),
17  );
18  expect(chatService.createCompletion.mock.calls[1][0].messages).toEqual(expect.arrayContaining([
19    expect.objectContaining({ role: 'tool', tool_call_id: 'call-delegate', content: expect.stringContaining('事实无误'), name: 'delegate_agent' }),
20  ]));
21  });
22
23  it('records malformed tool arguments as failed observations without invoking the tool', async () => {
24    const toolCall = {
25      id: 'call-bad',
26      type: 'function',
27      function: { name: 'calculator', arguments: '{"expression":' },
28    };
29    const invoke = jest.fn();
30    const { service } = makeService({ completions: [completion("", [toolCall]), completion('参数有误, 无法计算')], invoke });
31    const steps: Array<{ stepType: string; status: string; error: string }> = [];
32
33    const output = await (service as any).runToolUseHarness(
34      'user-1',
35      agent,
36      { input: '计算' },
37      'run-1',
38      { model: 'qwen-plus', messages: [{ role: 'user', content: '计算' }] },
39      [makeTool()],
40      [],
41      3,
42      async (step: any) => { steps.push(step); return step; },
43      completeStep,
44      { prompt_tokens: 0, completion_tokens: 0, total_tokens: 0 },
45    );
46
47    expect(output).toBe('参数有误, 无法计算');
48    expect(invoke).not.toHaveBeenCalled();
49    expect(steps).toEqual(expect.arrayContaining([
50      expect.objectContaining({ stepType: 'tool_call', status: 'failed', error: expect.stringContaining('JSON 解析失败') }),

```

```

1   expect.objectContaining({ stepType: 'observation', status: 'failed' }),
2   ));
3   });
4
5   it('does not expose disabled, high-risk, or unapproved confirm tools', async () => {
6     const allowedConfirm = makeTool({ id: 'confirm-1', name: 'safe_confirm', permissionLevel: 'confirm' });
7     const disabled = makeTool({ id: 'disabled-1', name: 'disabled_tool', permissionLevel: 'disabled' });
8     const highRisk = makeTool({ id: 'danger-1', name: 'external_delete', riskLevel: 'high' });
9     const getAgentTools = jest.fn(async () => [allowedConfirm, disabled, highRisk]);
10    const { service } = makeService({ getAgentTools });
11
12    const resolved = await (service as any).resolveRunnableTools(
13      'user-1',
14      { ...agent, source: 'user' },
15      ['confirm-1'],
16    );
17
18    expect(resolved.allowed.map((tool: ToolDefinition) => tool.id)).toEqual(['confirm-1']);
19    expect(resolved.skipped.map((item: { tool: ToolDefinition }) => item.tool.id)).toEqual(['disabled-1', 'danger-1']);
20  });
21
22  it('stops tool looping at maxSteps and asks for a final non-tool response', async () => {
23    const toolCall = {
24      id: 'call-1',
25      type: 'function',
26      function: { name: 'calculator', arguments: '{"expression":"1+2"}' },
27    };
28    const { service, chatService } = makeService({
29      completions: [completion("", [toolCall]), completion('达到上限，结果是 3')],
30    });
31
32    const output = await (service as any).runToolUseHarness(
33      'user-1',
34      agent,
35      { input: '计算 1+2' },
36      'run-1',
37      { model: 'qwen-plus', messages: [{ role: 'user', content: '计算 1+2' }] },
38      [makeTool()],
39      [],
40      1,
41      async (step: unknown) => step,
42      completeStep,
43      { prompt_tokens: 0, completion_tokens: 0, total_tokens: 0 },
44    );
45
46    expect(output).toBe('达到上限，结果是 3');
47    expect(chatService.createCompletion).toHaveBeenCalledTimes(2);
48    expect(chatService.createCompletion.mock.calls[1][0].tool_choice).toBe('none');
49    expect(chatService.createCompletion.mock.calls[1][0].tools).toBeUndefined();
50  });

```

```

1  });
2
3  describe('AgentsService run evaluation', () => {
4    it('scores a run with rules mode without calling an LLM judge', async () => {
5      const { service, chatService } = makeService();
6
7      const result = await (service as any).scoreRunWithJudge(makeRun(), { mode: 'rules' });
8
9      expect(result.score).toBeGreaterThan(0);
10     expect(result.rubric.mode).toBe('rules');
11     expect(chatService.createCompletion).not.toHaveBeenCalled();
12   });
13
14   it('falls back to rules scoring when the LLM judge fails', async () => {
15     const { service, chatService } = makeService();
16     chatService.createCompletion.mockRejectedValueOnce(new Error('judge timeout'));
17
18     const result = await (service as any).scoreRunWithJudge(makeRun(), { mode: 'hybrid', judgeModel: 'qwen-plus' });
19
20     expect(result.score).toBeGreaterThan(0);
21     expect(result.summary).toContain('LLM Judge 未完成');
22     expect(result.rubric.mode).toBe('rules_fallback');
23   });
24 });
25 // ===== backend/scripts/export-sanitized-sql.cjs =====
26 #!/usr/bin/env node
27
28 const fs = require('fs');
29 const path = require('path');
30 const dotenv = require('dotenv');
31 const mysql = require('mysql2/promise');
32
33 const backendDir = path.resolve(__dirname, '..');
34 const repoRoot = path.resolve(backendDir, '..');
35 const defaultBackupDir = path.join(backendDir, 'sqlbackup');
36
37 function loadEnvFiles() {
38   const files = [
39     path.join(backendDir, '.env'),
40     path.join(repoRoot, '.env'),
41     path.join(backendDir, 'sqlbackup', '.env'),
42     path.join(repoRoot, '.env.docker'),
43   ];
44
45   for (const file of files) {
46     if (fs.existsSync(file)) {
47       dotenv.config({ path: file, override: false });
48     }
49   }
50 }

```

```

1
2 function parseArgs(argv) {
3   const options = {
4     backupDir: process.env.BACKUP_DIR || "",
5     output: "",
6     keepDays: Number(process.env.RETAIN_DAYS || 0),
7     help: false,
8   };
9
10  for (const arg of argv) {
11    if (arg === '-h' || arg === '--help') options.help = true;
12    else if (arg.startsWith('--backup-dir=')) options.backupDir = arg.slice('--backup-dir='.length);
13    else if (arg.startsWith('--out=')) options.output = arg.slice('--out='.length);
14    else if (arg.startsWith('--keep-days=')) options.keepDays = Number(arg.slice('--keep-days='.length));
15    else {
16      throw new Error(`未知参数: ${arg}`);
17    }
18  }
19
20  return options;
21 }
22
23 function printHelp() {
24   console.log(`用法:
25   node backend/scripts/export-sanitized-sql.cjs [--backup-dir=backend/sqlbackup] [--out=/path/to/file.sql] [--keep-days=30]
26
27   环境变量:
28   MYSQL_HOST / MYSQL_PORT / MYSQL_USER / MYSQL_PASSWORD / MYSQL_DATABASE
29   兼容 DB_HOST / DB_PORT / DB_USER / DB_PASSWORD / DB_NAME
30   BACKUP_DIR    输出目录, 默认 backend/sqlbackup
31   RETAIN_DAYS    可选, 清理多少天前的 sanitized SQL 备份
32
33   说明:
34   导出完整表结构和数据, 但会自动屏蔽 API Key、Token、Secret 等敏感字段。`);
35 }
36
37 function localTimestamp() {
38   const d = new Date();
39   const pad = (n, size = 2) => String(n).padStart(size, '0');
40   return `${d.getFullYear()}${pad(d.getMonth() + 1)}${pad(d.getDate())}_${pad(d.getHours())}${pad(d.getMinutes())}${pad(d.getSeconds())}`;
41 }
42
43 function resolveOutputPath(options, database) {
44   if (options.output) {
45     return path.resolve(process.cwd(), options.output);
46   }
47
48   const backupDir = options.backupDir
49     ? path.resolve(process.cwd(), options.backupDir)
50     : defaultBackupDir;

```



```

1  const safeDb = String(database || 'database').replace(/[^a-zA-Z0-9_-]/g, '_');
2  return path.join(backupDir, `${safeDb}_sanitized_backup_${localTimestamp()}.sql`);
3  }
4
5  function getBaseConfig() {
6  return {
7    host: process.env.MYSQL_HOST || process.env.DB_HOST || '127.0.0.1',
8    port: Number(process.env.MYSQL_PORT || process.env.DB_PORT || 3306),
9    user: process.env.MYSQL_USER || process.env.DB_USER || 'root',
10   password: process.env.MYSQL_PASSWORD || process.env.MYSQL_ROOT_PASSWORD || process.env.DB_PASSWORD || '',
11   database: process.env.MYSQL_DATABASE || process.env.DB_NAME || 'llmgather',
12  };
13  }
14
15  function connectionCandidates(base) {
16  const candidates = [
17    base,
18  ];
19
20  const devPort = Number(process.env.MYSQL_DEV_PORT || 3307);
21  candidates.push({ ...base, host: '127.0.0.1', port: devPort });
22  candidates.push({ ...base, host: '127.0.0.1', port: 3306 });
23
24  const seen = new Set();
25  return candidates.filter((item) => {
26    const key = `${item.host}:${item.port}:${item.user}:${item.database}`;
27    if (seen.has(key)) return false;
28    seen.add(key);
29    return true;
30  });
31  }
32
33  async function connectMysql(baseConfig) {
34  const errors = [];
35  for (const candidate of connectionCandidates(baseConfig)) {
36    try {
37      const conn = await mysql.createConnection({
38        ...candidate,
39        charset: 'utf8mb4',
40        dateStrings: true,
41        supportBigNumbers: true,
42        bigNumberStrings: true,
43      });
44      return { conn, config: candidate };
45    } catch (error) {
46      errors.push(`${candidate.host}:${candidate.port} -> ${error.message}`);
47    }
48  }
49
50  throw new Error(`无法连接 MySQL。已尝试:\n${errors.map((item) => ` - ${item}`)}.join("\n")`);

```

```

1 }
2
3 function quoteIdent(identifier) {
4   return `${String(identifier).replace(/`/g, '`')}`;
5 }
6
7 function sqlValue(value) {
8   if (value === null || value === undefined) return 'NULL';
9   if (Buffer.isBuffer(value)) return `X${value.toString('hex')}`;
10  if (typeof value === 'number') return Number.isFinite(value) ? String(value) : 'NULL';
11  if (typeof value === 'bigint') return String(value);
12  if (typeof value === 'boolean') return value ? '1' : '0';
13
14  const text = String(value)
15    .replace(/\0/g, '\\0')
16    .replace(/\\/g, '\\\\')
17    .replace(/\n/g, '\\n')
18    .replace(/\r/g, '\\r')
19    .replace(/\x1a/g, '\\Z')
20    .replace(/'/g, '""');
21  return `${text}`;
22 }
23
24 const sensitiveJsonKeyPattern = /^(apiKey|api_key|api-key|key|token|accessToken|access_token|access-token|refreshToken|refresh_token|refresh-token|
25   secret|clientSecret|client_secret|client-secret|password|privateKey|private_key|private-key)$/i;
26
27 function isSensitiveJsonKey(key) {
28   return sensitiveJsonKeyPattern.test(String(key));
29 }
30
31 function redactValue(label) {
32   return `__REDACTED_${label.toUpperCase()}__`;
33 }
34
35 function sanitizeJsonValue(value, stats) {
36   if (Array.isArray(value)) {
37     return value.map((item) => sanitizeJsonValue(item, stats));
38   }
39   if (value && typeof value === 'object') {
40     const next = {};
41     for (const [key, item] of Object.entries(value)) {
42       if (isSensitiveJsonKey(key)) {
43         next[key] = redactValue('secret');
44         stats.json += 1;
45       } else {
46         next[key] = sanitizeJsonValue(item, stats);
47       }
48     }
49     return next;
50   }

```

```

1  return value;
2  }
3
4  function maybeSanitizeJson(text, stats) {
5    if (typeof text !== 'string') return text;
6    const trimmed = text.trim();
7    if (!trimmed || (!trimmed.startsWith('{') && !trimmed.startsWith('['))) return text;
8
9    try {
10     const parsed = JSON.parse(trimmed);
11     const before = stats.json;
12     const sanitized = sanitizeJsonValue(parsed, stats);
13     if (stats.json === before) return text;
14     return JSON.stringify(sanitized);
15   } catch {
16     return text;
17   }
18 }
19
20 function sanitizeColumnValue(table, column, value, stats) {
21   const tableName = String(table).toLowerCase();
22   const columnName = String(column).toLowerCase();
23
24   if (value === null || value === undefined) return value;
25
26   if (tableName === 'provider_api_keys' && columnName === 'api_key') {
27     stats.direct += 1;
28     return redactValue('api_key');
29   }
30
31   if (tableName === 'api_keys' && columnName === 'key_hash') {
32     stats.direct += 1;
33     return redactValue('api_key_hash');
34   }
35
36   if (
37     columnName === 'api_key' ||
38     columnName === 'apikey' ||
39     columnName === 'access_token' ||
40     columnName === 'refresh_token' ||
41     columnName === 'token' ||
42     columnName === 'secret' ||
43     columnName === 'private_key'
44   ) {
45     stats.direct += 1;
46     return redactValue(columnName.replace(/^[a-z0-9]+/g, '_'));
47   }
48
49   if (
50     columnName.endsWith('_json') ||
51     columnName === 'metadata' ||

```

```

1  columnName === 'audit_metadata'
2  ){
3    return maybeSanitizeJson(value, stats);
4  }
5
6  return value;
7 }
8
9 async function listTables(conn, database) {
10  const [rows] = await conn.query(
11    `SELECT TABLE_NAME AS tableName
12      FROM information_schema.TABLES
13     WHERE TABLE_SCHEMA = ? AND TABLE_TYPE = 'BASE TABLE'
14     ORDER BY TABLE_NAME ASC`,
15    [database],
16  );
17  return rows.map((row) => row.tableName);
18 }
19
20 async function showCreateTable(conn, table) {
21  const [rows] = await conn.query(`SHOW CREATE TABLE ${quoteIdent(table)}`);
22  return rows[0]['Create Table'];
23 }
24
25 async function listColumns(conn, table) {
26  const [rows] = await conn.query(`SHOW COLUMNS FROM ${quoteIdent(table)}`);
27  return rows.map((row) => row.Field);
28 }
29
30 async function dumpTableData(conn, stream, table, columns, stats) {
31  const [rows] = await conn.query(
32    `SELECT ${columns.map(quoteIdent).join(', ')} FROM ${quoteIdent(table)}`,
33  );
34
35  if (rows.length === 0) return 0;
36
37  const columnSql = columns.map(quoteIdent).join(', ');
38  for (const row of rows) {
39    const values = columns.map((column) => sqlValue(sanitizeColumnValue(table, column, row[column], stats)));
40    stream.write(`INSERT INTO ${quoteIdent(table)} (${columnSql}) VALUES (${values.join(', ')});\n`);
41  }
42  return rows.length;
43 }
44
45 function cleanupOldBackups(backupDir, database, keepDays) {
46  if (!Number.isFinite(keepDays) || keepDays <= 0) return 0;
47  if (!fs.existsSync(backupDir)) return 0;
48
49  const now = Date.now();
50  const maxAgeMs = keepDays * 24 * 60 * 60 * 1000;

```

```

1  const prefix = `${database}_sanitized_backup_`;
2  let removed = 0;
3
4  for (const name of fs.readdirSync(backupDir)) {
5    if (!name.startsWith(prefix) || !name.endsWith('.sql')) continue;
6    const fullPath = path.join(backupDir, name);
7    const stat = fs.statSync(fullPath);
8    if (now - stat.mtimeMs > maxAgeMs) {
9      fs.unlinkSync(fullPath);
10     removed += 1;
11   }
12 }
13 return removed;
14 }
15
16 async function main() {
17   loadEnvFiles();
18   const options = parseArgs(process.argv.slice(2));
19   if (options.help) {
20     printHelp();
21     return;
22   }
23
24   const baseConfig = getBaseConfig();
25   const outputPath = resolveOutputPath(options, baseConfig.database);
26   const backupDir = path.dirname(outputPath);
27   fs.mkdirSync(backupDir, { recursive: true });
28
29   const { conn, config } = await connectMysql(baseConfig);
30   const stats = { direct: 0, json: 0 };
31   let tableCount = 0;
32   let rowCount = 0;
33
34   const stream = fs.createWriteStream(outputPath, { encoding: 'utf8', mode: 0o600 });
35   try {
36     const tables = await listTables(conn, config.database);
37     if (tables.length === 0) {
38       throw new Error(`数据库 ${config.database} 中没有可导出的表`);
39     }
40
41     stream.write(`-- LLM Gather sanitized SQL backup\n`);
42     stream.write(`-- Database: ${config.database}\n`);
43     stream.write(`-- Created at: ${new Date().toISOString()}\n`);
44     stream.write(`-- Sensitive API keys/tokens/secrets are redacted.\n\n`);
45     stream.write(`SET NAMES utf8mb4;\n`);
46     stream.write(`SET FOREIGN_KEY_CHECKS=0;\n\n`);
47
48     for (const table of tables) {
49       tableCount += 1;
50       const createSql = await showCreateTable(conn, table);

```

```

1   const columns = await listColumns(conn, table);
2
3   stream.write(`--\n-- Table structure for ${quoteIdent(table)}\n--\n`);
4   stream.write(`DROP TABLE IF EXISTS ${quoteIdent(table)};\n`);
5   stream.write(`${createSql};\n\n`);
6
7   stream.write(`--\n-- Data for ${quoteIdent(table)}\n--\n`);
8   const rows = await dumpTableData(conn, stream, table, columns, stats);
9   rowCount += rows;
10  stream.write(`\n`);
11  }
12
13  stream.write(`SET FOREIGN_KEY_CHECKS=1;\n`);
14  } finally {
15    await new Promise((resolve, reject) => {
16      stream.end((error) => error ? reject(error) : resolve());
17    });
18    await conn.end();
19  }
20
21  const stat = fs.statSync(outputPath);
22  if (stat.size < 100) {
23    fs.unlinkSync(outputPath);
24    throw new Error('备份文件过小，已删除异常输出');
25  }
26
27  const removed = cleanupOldBackups(backupDir, config.database, options.keepDays);
28  console.log(`数据库已导出: ${outputPath}`);
29  console.log(`表: ${tableCount}, 行: ${rowCount}, 已脱敏字段: ${stats.direct + stats.json} (列 ${stats.direct}, JSON ${stats.json})`);
30  if (removed > 0) console.log(`已清理旧备份: ${removed} 个`);
31  }
32
33  main().catch((error) => {
34    console.error(`导出失败: ${error.message}`);
35    process.exit(1);
36  });
37  // ===== backend/src/providers/gemini.provider.spec.ts =====
38  import { ApiKeyPool } from './api-key-pool';
39  import { GeminiProvider } from './gemini.provider';
40
41  function makeProvider(options: { keys?: string[]; retryCount?: number } = {}): GeminiProvider {
42    const provider = new GeminiProvider({
43      providerName: 'gemini',
44      baseUrl: 'https://generativelanguage.googleapis.com/v1beta',
45      models: ['gemini-3.5-flash'],
46      retryCount: options.retryCount ?? 0,
47    });
48    provider.setKeyPool(new ApiKeyPool(options.keys ?? ['gemini-test-key']));
49    return provider;
50  }

```

```

1
2 describe('GeminiProvider', () => {
3   const originalFetch = global.fetch;
4
5   afterEach(() => {
6     global.fetch = originalFetch;
7     jest.restoreAllMocks();
8   });
9
10  it('matches configured models and models/ prefixed ids', () => {
11    const provider = makeProvider();
12    expect(provider.isModelSupported('gemini-3.5-flash')).toBe(true);
13    expect(provider.isModelSupported('models/gemini-3.5-flash')).toBe(true);
14    expect(provider.isModelSupported('gemini-2.5-flash')).toBe(false);
15    expect(provider.isModelSupported('qwen-plus')).toBe(false);
16  });
17
18  it('converts OpenAI style chat requests to Gemini generateContent requests', async () => {
19    const fetchMock = jest.fn().mockResolvedValue(
20      new Response(
21        JSON.stringify({
22          candidates: [{ content: { parts: [{ text: 'hello from gemini' }] } }, finishReason: 'STOP' ]},
23          usageMetadata: { promptTokenCount: 3, candidatesTokenCount: 4, totalTokenCount: 7 },
24        }),
25        { status: 200 },
26      ),
27    );
28    global.fetch = fetchMock;
29
30    const provider = makeProvider();
31    const result = await provider.chatCompletion({
32      model: 'gemini-3.5-flash',
33      temperature: 0.3,
34      top_p: 0.8,
35      max_tokens: 128,
36      messages: [
37        { role: 'system', content: 'Be concise.' },
38        {
39          role: 'user',
40          content: [
41            { type: 'text', text: 'Describe this image.' },
42            { type: 'image_url', image_url: { url: '' } },
43          ],
44        },
45      ],
46    });
47
48    expect(fetchMock).toHaveBeenCalledTimes(1);
49    const [url, init] = fetchMock.mock.calls[0] as [string, RequestInit];
50    expect(url).toBe(

```

```

1  'https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:generateContent',
2  );
3  expect(init.method).toBe('POST');
4  expect(init.headers).toMatchObject({
5    'Content-Type': 'application/json',
6    'x-goog-api-key': 'gemini-test-key',
7  });
8
9  const body = JSON.parse(String(init.body));
10 expect(body).toMatchObject({
11   systemInstruction: { parts: [{ text: 'Be concise.' }] },
12   generationConfig: { temperature: 0.3, topP: 0.8, maxOutputTokens: 128 },
13   contents: [
14     {
15       role: 'user',
16       parts: [
17         { text: 'Describe this image.' },
18         { inlineData: { mimeType: 'image/png', data: 'abc123' } },
19       ],
20     },
21   ],
22 });
23 expect(result.choices[0].message.content).toBe('hello from gemini');
24 expect(result.choices[0].finish_reason).toBe('stop');
25 expect(result.usage).toEqual({ prompt_tokens: 3, completion_tokens: 4, total_tokens: 7 });
26 });
27
28 it('maps multi-turn messages, thinking config, safety, structured output, and tools', async () => {
29   const fetchMock = jest.fn().mockResolvedValue(
30     new Response(
31       JSON.stringify({
32         candidates: [{ content: { parts: [{ text: 'done' }] }, finishReason: 'STOP' }],
33       }),
34       { status: 200 },
35     ),
36   );
37   global.fetch = fetchMock;
38
39   await makeProvider().chatCompletion({
40     model: 'gemini-3.5-flash',
41     messages: [
42       { role: 'system', content: 'System prompt' },
43       { role: 'user', content: 'Hello' },
44       { role: 'assistant', content: 'Hi there' },
45       { role: 'user', content: 'Use a tool if needed' },
46     ],
47     tools: [{
48       type: 'function',
49       function: {
50         name: 'lookup',

```



```

1     description: 'Lookup data',
2     parameters: { type: 'object', properties: { q: { type: 'string' } } },
3   },
4 },
5 tool_choice: { type: 'function', function: { name: 'lookup' } },
6 extra_body: {
7   thinking_config: { thinking_level: 'medium', include_thoughts: true },
8   safety_settings: [{ category: 'HARM_CATEGORY_HARASSMENT', threshold: 'BLOCK_MEDIUM_AND_ABOVE' }],
9   response_mime_type: 'application/json',
10  response_schema: { type: 'object', properties: { ok: { type: 'boolean' } } },
11  cached_content: 'cachedContents/test-cache',
12 },
13 });
14
15 const [, init] = fetchMock.mock.calls[0] as [string, RequestInit];
16 const body = JSON.parse(String(init.body));
17 expect(body).toMatchObject({
18   systemInstruction: { parts: [{ text: 'System prompt' }] },
19   contents: [
20     { role: 'user', parts: [{ text: 'Hello' }] },
21     { role: 'model', parts: [{ text: 'Hi there' }] },
22     { role: 'user', parts: [{ text: 'Use a tool if needed' }] },
23   ],
24   generationConfig: {
25     thinkingConfig: { thinkingLevel: 'medium', includeThoughts: true },
26     responseMimeType: 'application/json',
27     responseSchema: { type: 'object', properties: { ok: { type: 'boolean' } } },
28   },
29   safetySettings: [{ category: 'HARM_CATEGORY_HARASSMENT', threshold: 'BLOCK_MEDIUM_AND_ABOVE' }],
30   tools: [{
31     functionDeclarations: [{
32       name: 'lookup',
33       description: 'Lookup data',
34       parameters: { type: 'object', properties: { q: { type: 'string' } } },
35     }],
36   }],
37   toolConfig: {
38     functionCallingConfig: { mode: 'ANY', allowedFunctionNames: ['lookup'] },
39   },
40   cachedContent: 'cachedContents/test-cache',
41 });
42 expect(body.generationConfig.thinkingConfig.thinking_level).toBeUndefined();
43 expect(body.generationConfig.thinkingConfig.include_thoughts).toBeUndefined();
44 });
45
46 it('wraps Gemini SSE chunks as OpenAI-compatible stream chunks', async () => {
47   const fetchMock = jest.fn().mockResolvedValue(
48     new Response(
49       'data: {"candidates":[{"content":{"parts":[{"text":"hi"}]}}]\n\n',
50       { status: 200 },

```

```

1   ),
2   );
3   global.fetch = fetchMock;
4
5   const response = await makeProvider().chatCompletionStream({
6     model: 'gemini-3.5-flash',
7     messages: [{ role: 'user', content: 'hello' }],
8     stream: true,
9   });
10
11  const text = await response.text();
12  const [url, init] = fetchMock.mock.calls[0] as [string, RequestInit];
13  expect(url).toBe(
14    'https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-flash:streamGenerateContent?alt=sse',
15  );
16  expect(init.headers).toMatchObject({
17    'Content-Type': 'application/json',
18    'x-goog-api-key': 'gemini-test-key',
19    Accept: 'text/event-stream',
20  });
21  expect(text).toContain('"object":"chat.completion.chunk"');
22  expect(text).toContain('"content":"hi"');
23  expect(text).toContain('data: [DONE]');
24  });
25
26  it('rotates API keys when a Gemini stream request is rate limited before streaming starts', async () => {
27    const fetchMock = jest.fn()
28      .mockResolvedValueOnce(new Response(JSON.stringify({ error: { message: 'quota exceeded' } }), { status: 429 }))
29      .mockResolvedValueOnce(
30        new Response(
31          'data: {"candidates":[{"content":{"parts":[{"text":"hi"}]}]}\n\n',
32          { status: 200 },
33        ),
34      );
35    global.fetch = fetchMock;
36    jest.spyOn(Math, 'random').mockReturnValue(0);
37
38    const response = await makeProvider({ keys: ['gemini-key-a', 'gemini-key-b'], retryCount: 1 }).chatCompletionStream({
39      model: 'gemini-3.5-flash',
40      messages: [{ role: 'user', content: 'hello' }],
41      stream: true,
42    });
43
44    const text = await response.text();
45    const firstCall = fetchMock.mock.calls[0][1] as RequestInit;
46    const secondCall = fetchMock.mock.calls[1][1] as RequestInit;
47    expect(fetchMock).toHaveBeenCalledTimes(2);
48    expect(firstCall.headers).toMatchObject({ 'x-goog-api-key': 'gemini-key-a' });
49    expect(secondCall.headers).toMatchObject({ 'x-goog-api-key': 'gemini-key-b' });
50    expect(JSON.parse(response.headers.get('x-provider-key-rotation') || '{}')).toEqual({

```

```

1   provider: 'gemini',
2   attempts: 1,
3   reason: 'rate_limit',
4   });
5   expect(text).toContain('"content":"hi"');
6   });
7
8   it('rotates API keys when a Gemini key has insufficient balance', async () => {
9     const fetchMock = jest.fn()
10    .mockResolvedValueOnce(new Response(JSON.stringify({ error: { message: 'insufficient balance' } }), { status: 402 }))
11    .mockResolvedValueOnce(
12      new Response(
13        JSON.stringify({ candidates: [{ content: { parts: [{ text: 'funded key ok' } ] }, finishReason: 'STOP' } ] }),
14        { status: 200 },
15      ),
16    );
17    global.fetch = fetchMock;
18    jest.spyOn(Math, 'random').mockReturnValue(0);
19
20    const response = await makeProvider({ keys: ['gemini-empty', 'gemini-funded'], retryCount: 1 }).chatCompletion({
21      model: 'gemini-3.5-flash',
22      messages: [{ role: 'user', content: 'hello' }],
23    });
24
25    const firstCall = fetchMock.mock.calls[0][1] as RequestInit;
26    const secondCall = fetchMock.mock.calls[1][1] as RequestInit;
27    expect(fetchMock).toHaveBeenCalledTimes(2);
28    expect(firstCall.headers).toMatchObject({ 'x-goog-api-key': 'gemini-empty' });
29    expect(secondCall.headers).toMatchObject({ 'x-goog-api-key': 'gemini-funded' });
30    expect(response.choices[0].message.content).toBe('funded key ok');
31    expect((response as any)._providerKeyRotation).toEqual({
32      provider: 'gemini',
33      attempts: 1,
34      reason: 'balance_exhausted',
35    });
36  });
37
38  it('emits Gemini thought summary parts as reasoning_content in streams', async () => {
39    global.fetch = jest.fn().mockResolvedValue(
40      new Response(
41        'data: {"candidates":[{"content":{"parts":[{"text":"Thinking..."thought":true}, {"text":"Answer."}]}]}\n\n',
42        { status: 200 },
43      ),
44    );
45
46    const response = await makeProvider().chatCompletionStream({
47      model: 'gemini-3.5-flash',
48      messages: [{ role: 'user', content: 'reason' }],
49      stream: true,
50      extra_body: { enable_thinking: true },

```

```

1  });
2
3  const text = await response.text();
4  expect(text).toContain("reasoning_content":"Thinking...");
5  expect(text).toContain("content":"Answer.");
6  const [, init] = (global.fetch as jest.Mock).mock.calls[0] as [string, RequestInit];
7  const body = JSON.parse(String(init.body));
8  expect(body.generationConfig.thinkingConfig).toEqual({ includeThoughts: true });
9  });
10
11 it('parses CRLF SSE chunks and trailing events without a blank-line terminator', async () => {
12   global.fetch = jest.fn().mockResolvedValue(
13     new Response(
14       'data: {"candidates":[{"content":{"parts":[{"text":"hello"}]}}}\r\n\r\n' +
15       'data: {"candidates":[{"content":{"parts":[{"text":" world"}]}}}',
16       { status: 200 },
17     ),
18   );
19
20   const response = await makeProvider().chatCompletionStream({
21     model: 'gemini-3.5-flash',
22     messages: [{ role: 'user', content: 'hello' }],
23     stream: true,
24   });
25
26   const text = await response.text();
27   expect(text).toContain("content":"hello");
28   expect(text).toContain("content":" world");
29   expect(text).not.toContain('returned an empty response');
30 });
31
32 it('parses raw JSON stream fragments when the provider omits data prefixes', async () => {
33   global.fetch = jest.fn().mockResolvedValue(
34     new Response(
35       '{"candidates":[{"content":{"parts":[{"text":"raw json"}]}}}',
36       { status: 200 },
37     ),
38   );
39
40   const response = await makeProvider().chatCompletionStream({
41     model: 'gemini-3.5-flash',
42     messages: [{ role: 'user', content: 'hello' }],
43     stream: true,
44   });
45
46   const text = await response.text();
47   expect(text).toContain("content":"raw json");
48   expect(text).not.toContain('returned an empty response');
49 });
50

```

```

1  it('throws a readable error when Gemini returns no text in non-stream mode', async () => {
2    global.fetch = jest.fn().mockResolvedValue(
3      new Response(
4        JSON.stringify({
5          candidates: [{ finishReason: 'SAFETY', safetyRatings: [{ category: 'HARM_CATEGORY_TEST', probability: 'HIGH' }] }],
6        }),
7        { status: 200 },
8      ),
9    );
10
11   await expect(makeProvider().chatCompletion({
12     model: 'gemini-3.5-flash',
13     messages: [{ role: 'user', content: 'hello' }],
14   })).rejects.toThrow('finish reason: SAFETY');
15 });
16
17 it('surfaces Gemini provider error details from non-JSON responses', async () => {
18   global.fetch = jest.fn().mockResolvedValue(
19     new Response('upstream html error', { status: 500, statusText: 'Internal Server Error' }),
20   );
21
22   await expect(makeProvider().chatCompletion({
23     model: 'gemini-3.5-flash',
24     messages: [{ role: 'user', content: 'hello' }],
25   })).rejects.toThrow('upstream html error');
26 });
27
28 it('emits an explanatory stream chunk when Gemini stream has no text', async () => {
29   global.fetch = jest.fn().mockResolvedValue(
30     new Response(
31       'data: {"candidates":[{"finishReason":"SAFETY"}]}\n\n',
32       { status: 200 },
33     ),
34   );
35
36   const response = await makeProvider().chatCompletionStream({
37     model: 'gemini-3.5-flash',
38     messages: [{ role: 'user', content: 'hello' }],
39     stream: true,
40   });
41
42   const text = await response.text();
43   expect(text).toContain('gemini returned no text');
44   expect(text).toContain('finish reason: SAFETY');
45   expect(text).toContain('data: [DONE]');
46 });
47 });
48 // ===== backend/src/memory/providers/langgraph-memory.provider.ts =====
49 import { BadRequestException, Injectable, Logger } from '@nestjs/common';
50 import { CreateMemoryDto, UpdateMemoryDto } from '../dto/memory.dto';

```

```

1 import { MemoryProvider } from './memory-provider';
2 import { MemoryItem, MEMORY_TYPES } from './memory.types';
3 import { normalizeMemoryType } from './memory-utils';
4 import { NativeMemoryProvider } from './native-memory.provider';
5
6 type LangGraphStoreValue = {
7   content: string;
8   memoryType: string;
9   importance: number;
10  userId: string;
11  agentId: string | null;
12  namespace: string;
13  metadata?: Record<string, unknown>;
14 };
15
16 type LangGraphMemoryGraphState = {
17   status?: string;
18   error?: string;
19   key?: string;
20   result?: unknown;
21   results?: unknown[];
22   namespacePath?: string[];
23   value?: LangGraphStoreValue;
24 };
25
26 @Injectable()
27 export class LangGraphMemoryProvider implements MemoryProvider {
28   private readonly logger = new Logger(LangGraphMemoryProvider.name);
29
30   constructor(private readonly nativeProvider: NativeMemoryProvider) {}
31
32   capabilities() {
33     const configured = Boolean(this.baseUrl);
34     return {
35       provider: 'langgraph' as const,
36       displayName: 'LangGraph Memory',
37       configured,
38       writable: configured,
39       searchable: configured,
40       supportedTypes: MEMORY_TYPES,
41       note: configured ? '通过 LangGraph Store 同步，并保留本地镜像' : '需要配置 LANGGRAPH_MEMORY_URL',
42     };
43   }
44
45   async list(userId: string, agentId?: string): Promise<MemoryItem[]> {
46     return this.nativeProvider.list(userId, agentId);
47   }
48
49   async create(userId: string, dto: CreateMemoryDto): Promise<MemoryItem> {
50     this.assertConfigured();

```

```

1  const memoryType = normalizeMemoryType(dto.memoryType);
2  const namespace = dto.namespace?.trim() || 'default';
3  const agentId = dto.agentId ?? null;
4  const key = this.key(userId, agentId, namespace, memoryType);
5  const value: LangGraphStoreValue = {
6    content: dto.content.trim(),
7    memoryType,
8    importance: dto.importance ?? 3,
9    userId,
10   agentId,
11   namespace,
12   metadata: dto.metadata,
13  };
14  const namespacePath = this.namespace(userId, agentId, namespace);
15  const payload = await this.runMemoryGraphWithStoreFallback(
16    {
17      operation: 'put',
18      userId,
19      agentId,
20      namespace,
21      namespacePath,
22      key,
23      value,
24    },
25    () => this.putStoreItem(namespacePath, key, value),
26  );
27  const externalId = this.externalId(payload) || key;
28  return this.nativeProvider.createMirror(
29    userId,
30    { ...dto, memoryType },
31    'langgraph',
32    externalId,
33    JSON.stringify(payload ?? value),
34  );
35 }
36
37 async search(userId: string, query: string, agentId?: string, limit = 5): Promise<MemoryItem[]> {
38   if (!this.baseUrl) {
39     return this.nativeProvider.search(userId, query, agentId, limit);
40   }
41   try {
42     const graphState = await this.runMemoryGraph({
43       operation: 'search',
44       userId,
45       agentId: agentId ?? null,
46       query,
47       limit,
48       includeGlobal: true,
49     });
50     const results = graphState.results ?? [];

```

```

1  const remoteMemories = this.parseSearchResults(userId, agentId, results);
2  if (remoteMemories.length > 0) return remoteMemories.slice(0, limit);
3  } catch (error) {
4    this.logger.warn(`LangGraph memory graph search failed, fallback to Store/local mirror: ${this.errorMessage(error)}`);
5    try {
6      const results = await this.searchStoreForAgent(userId, agentId ?? null, query, limit);
7      const remoteMemories = this.parseSearchResults(userId, agentId, results);
8      if (remoteMemories.length > 0) return remoteMemories.slice(0, limit);
9    } catch (storeError) {
10     this.logger.warn(`LangGraph memory Store search failed, fallback to local mirror: ${this.errorMessage(storeError)}`);
11   }
12 }
13 return this.nativeProvider.search(userId, query, agentId, limit);
14 }
15
16 async update(userId: string, id: string, dto: UpdateMemoryDto): Promise<MemoryItem> {
17   this.assertConfigured();
18   const current = await this.nativeProvider.getOwned(userId, id);
19   const memoryType = normalizeMemoryType(dto.memoryType ?? current.memoryType);
20   const namespace = dto.namespace?.trim() || current.namespace;
21   const agentId = current.agentId;
22   const key = current.externalId || this.key(userId, agentId, namespace, memoryType);
23   const value: LangGraphStoreValue = {
24     content: dto.content?.trim() || current.content,
25     memoryType,
26     importance: dto.importance ?? current.importance,
27     userId,
28     agentId,
29     namespace,
30     metadata: dto.metadata,
31   };
32   const namespacePath = this.namespace(userId, agentId, namespace);
33   const payload = await this.runMemoryGraphWithStoreFallback(
34     {
35       operation: 'put',
36       userId,
37       agentId,
38       namespace,
39       namespacePath,
40       key,
41       value,
42     },
43     () => this.putStoreItem(namespacePath, key, value),
44   );
45   return this.nativeProvider.updateProviderMirror(
46     userId,
47     id,
48     { ...dto, memoryType, content: value.content, importance: value.importance, namespace },
49     key,
50     JSON.stringify(payload ?? value),

```



```

1  );
2  }
3
4  async remove(userId: string, id: string): Promise<void> {
5    const current = await this.nativeProvider.getOwned(userId, id);
6    if (this.baseUrl && current.externalId) {
7      try {
8        const namespacePath = this.namespace(userId, current.agentId, current.namespace);
9        await this.runMemoryGraphWithStoreFallback(
10         {
11           operation: 'delete',
12           userId,
13           agentId: current.agentId,
14           namespace: current.namespace,
15           namespacePath,
16           key: current.externalId,
17         },
18         () => this.deleteStoreItem(namespacePath, current.externalId),
19       );
20     } catch (error) {
21       this.logger.warn(`LangGraph memory delete failed: ${this.errorMessage(error)}`);
22     }
23   }
24   await this.nativeProvider.remove(userId, id);
25 }
26
27 async removeAll(userId: string, agentId?: string): Promise<number> {
28   return this.nativeProvider.removeAll(userId, agentId);
29 }
30
31 private get baseUrl(): string {
32   return (process.env.LANGGRAPH_MEMORY_URL || process.env.LANGGRAPH_API_URL || "").trim().replace(/\/+$/, "");
33 }
34
35 private get apiKey(): string {
36   return (process.env.LANGGRAPH_MEMORY_API_KEY || process.env.LANGGRAPH_API_KEY || "").trim();
37 }
38
39 private get assistantId(): string {
40   return (process.env.LANGGRAPH_MEMORY_ASSISTANT_ID || 'memory').trim();
41 }
42
43 private assertConfigured(): void {
44   if (!this.baseUrl) {
45     throw new BadRequestException('LangGraph Memory 未配置, 请设置 LANGGRAPH_MEMORY_URL');
46   }
47 }
48
49 private namespace(userId: string, agentId: string | null | undefined, namespace: string): string[] {
50   return ['llmgather', userId, agentId || 'global', namespace || 'default'];

```

```

1  }
2
3  private key(userId: string, agentId: string | null | undefined, namespace: string, memoryType: string): string {
4    const suffix = `${Date.now()}-${Math.random().toString(16).slice(2, 10)}`;
5    return `${userId}:${agentId || 'global'}:${namespace}:${memoryType}:${suffix}`;
6  }
7
8  private async runMemoryGraph(input: Record<string, unknown>): Promise<LangGraphMemoryGraphState> {
9    const payload = await this.request('/runs/wait', {
10      method: 'POST',
11      body: JSON.stringify({
12        assistant_id: this.assistantId,
13        input,
14        metadata: {
15          source: 'llmgather-memory-provider',
16          operation: input.operation,
17        },
18      }),
19    });
20    const state = this.unwrapGraphState(payload);
21    if (state.status === 'failed' || state.error) {
22      throw new BadRequestException(`LangGraph Memory 图执行失败: ${state.error || state.status}`);
23    }
24    return state;
25  }
26
27  private async runMemoryGraphWithStoreFallback(input: Record<string, unknown>, fallback: () => Promise<unknown>): Promise<unknown> {
28    try {
29      const state = await this.runMemoryGraph(input);
30      return state.result ?? state;
31    } catch (error) {
32      this.logger.warn(`LangGraph memory graph failed, fallback to Store REST: ${this.errorMessage(error)}`);
33      return fallback();
34    }
35  }
36
37  private async putStoreItem(namespace: string[], key: string, value: LangGraphStoreValue): Promise<unknown> {
38    return this.request('/store/items', {
39      method: 'PUT',
40      body: JSON.stringify({ namespace, key, value }),
41    });
42  }
43
44  private async searchStoreForAgent(userId: string, agentId: string | null, query: string, limit: number): Promise<unknown[]> {
45    const prefixes = [this.namespace(userId, agentId, 'default').slice(0, 3)];
46    if (agentId) prefixes.push(this.namespace(userId, null, 'default').slice(0, 3));
47    const groups = await Promise.all(prefixes.map((prefix) => this.searchStore(prefix, query, limit)));
48    const seen = new Set<string>();
49    const merged: unknown[] = [];
50    for (const group of groups) {

```

```

1  for (const item of this.searchItems(group)) {
2      const record = item as Record<string, unknown>;
3      const id = `${Array.isArray(record.namespace) ? record.namespace.join('/') : ''}${String(record.key || record.id || '')}`;
4      if (seen.has(id)) continue;
5      seen.add(id);
6      merged.push(item);
7  }
8  }
9  return merged.slice(0, limit);
10 }
11
12 private async searchStore(namespace: string[], query: string, limit: number): Promise<unknown> {
13     return this.request('/store/items/search', {
14         method: 'POST',
15         body: JSON.stringify({ namespace_prefix: namespace.slice(0, 3), query, limit }),
16     });
17 }
18
19 private async deleteStoreItem(namespace: string[], key: string): Promise<unknown> {
20     return this.request('/store/items', {
21         method: 'DELETE',
22         body: JSON.stringify({ namespace, key }),
23     });
24 }
25
26 private async request(path: string, init: RequestInit): Promise<unknown> {
27     const response = await fetch(`${this.baseUrl}${path}`, {
28         ...init,
29         headers: {
30             'Content-Type': 'application/json',
31             ...(this.apiKey ? { Authorization: `Bearer ${this.apiKey}` } : {}),
32             ...(init.headers ?? {}),
33         },
34     });
35     const text = await response.text();
36     let payload: unknown = undefined;
37     if (text) {
38         try { payload = JSON.parse(text); } catch { payload = text; }
39     }
40     if (!response.ok) {
41         throw new BadRequestException(`LangGraph Memory 请求失败 (${response.status}): ${typeof payload === 'string' ? payload : JSON.stringify(payload)}`);
42     }
43     return payload;
44 }
45
46 private externalId(payload: unknown): string {
47     if (!payload || typeof payload !== 'object') return '';
48     const record = payload as Record<string, unknown>;
49     return String(record.key || record.id || record.itemId || '');
50 }

```

```

1
2 private unwrapGraphState(payload: unknown): LangGraphMemoryGraphState {
3   if (!payload || typeof payload !== 'object') return {};
4   const record = payload as Record<string, unknown>;
5   const values = record.values && typeof record.values === 'object'
6     ? record.values as Record<string, unknown>
7     : record;
8   return values as LangGraphMemoryGraphState;
9 }
10
11 private searchItems(payload: unknown): unknown[] {
12   if (Array.isArray(payload)) return payload;
13   if (payload && typeof payload === 'object' && Array.isArray((payload as Record<string, unknown>).items)) {
14     return (payload as { items: unknown[] }).items;
15   }
16   return [];
17 }
18
19 private errorMessage(error: unknown): string {
20   return error instanceof Error ? error.message : String(error);
21 }
22
23 private parseSearchResults(userId: string, agentId: string | undefined, payload: unknown): MemoryItem[] {
24   const items = this.searchItems(payload);
25   return items.map((item, index) => {
26     const record = item as Record<string, unknown>;
27     const value = (record.value && typeof record.value === 'object' ? record.value : record) as Record<string, unknown>;
28     const namespace = Array.isArray(record.namespace) ? record.namespace.map(String) : [];
29     const remoteAgentId = namespace.length >= 3
30       ? namespace[2] !== 'global' ? namespace[2] : null
31       : agentId || null;
32     return {
33       id: String(record.key || record.id || `langgraph-${index}`),
34       userId,
35       agentId: remoteAgentId,
36       namespace: namespace.length > 0 ? String(namespace.at(-1) || 'default') : 'default',
37       memoryType: normalizeMemoryType(String(value.memoryType || 'fact')),
38       content: String(value.content || value.text || ''),
39       importance: Number(value.importance ?? 3),
40       metadata: JSON.stringify(value.metadata || {}),
41       provider: 'langgraph' as const,
42       externalId: String(record.key || record.id || ''),
43       providerPayload: JSON.stringify(item),
44       createdAt: String(record.created_at || record.createdAt || new Date().toISOString()),
45       updatedAt: String(record.updated_at || record.updatedAt || new Date().toISOString()),
46       score: Number(record.score ?? 0),
47     };
48   }).filter((item) => item.content.trim());
49 }
50 }

```